


Université Saint-Joseph de Beyrouth
Faculté d'ingénierie
Institut national des télécommunications
et de l'informatique

ACADEMIC PROGRAM

Master en Systèmes et Réseaux: Option Sécurité de l'Information

---

# ADMINISTRATION UNIX: MIDTERM PROJECT

## Falco Runtime Detection Rules

---

PREPARED BY

**Tony Doumit**

SUPERVISED BY

**Prof. Fadi Chebli**

Academic Year 2026 – 2027

October 2026

# Contents

**1 Objective** **4**

1.1 Deliverables . . . . . 4

1.2 Constraints and safety . . . . . 4

**2 Lab environment** **4**

2.1 Environment summary . . . . . 4

2.2 Why a virtual machine . . . . . 4

2.3 Why Ubuntu 24.04 LTS . . . . . 6

2.4 Network mode . . . . . 6

2.5 Snapshots . . . . . 6

**3 How Falco works** **6**

3.1 Flow of one event . . . . . 6

3.2 System calls and event details . . . . . 8

3.3 The driver . . . . . 8

3.4 The rules engine . . . . . 9

3.5 IDS versus IPS . . . . . 9

**4 Deployment** **10**

4.1 System preparation . . . . . 10

4.2 Environment readiness check . . . . . 10

4.3 Clipboard fix . . . . . 11

4.4 Connectivity check . . . . . 12

4.5 Falco repository . . . . . 12

4.6 Installation and verification . . . . . 13

**5 Compatibility analysis** **14**

5.1 The situation . . . . . 14

5.2 The reported problem . . . . . 14

5.3 Checks performed before relying on Falco . . . . . 14

5.4 Fallback plan . . . . . 15

5.5 Stability . . . . . 15

**6 Falco configuration** **15**

6.1 Rules files and loading order . . . . . 15

6.2 Observations at startup . . . . . 16

6.3 Automatic reload . . . . . 16

6.4 Settings not reviewed . . . . . 16

**7 Default rules review** **16**

7.1 Method used for each rule . . . . . 16

7.2 Where the default rules are . . . . . 18

7.3 Starting from a real alert . . . . . 18

7.4 Finding the rule behind the alert . . . . . 19

7.5 Anatomy of the rule *Read sensitive file untrusted* . . . . . 20

7.6 The definitions behind the condition . . . . . 22

7.7 Baseline before tuning . . . . . 22

7.8 Verifying the program . . . . . 24

7.9 Diagnosis . . . . . 24

7.10 Tuning applied . . . . . 25

**8 Custom rules** **28**

8.1 Custom rule A: setuid or setgid bit set on a file . . . . . 28

8.2 Custom rule B: a network scanner is started on the host . . . . . 36

8.3 Custom rule C: tampering with Falco . . . . . 42

8.4 Custom rule D: attack tools . . . . . 50

**9 Test scenarios** **56**

9.1 Scenario 1: reading a sensitive file . . . . . 56

9.2 Scenario 2: a shell started by a protected application . . . . . 57

9.3 Scenario 3: a program run from /dev/shm . . . . . 61

9.4 Scenario 4: the setuid bit (custom rule A) . . . . . 64

9.5 Scenario 5: a network scanner (custom rule B) . . . . . 64

9.6 Scenario 6: tampering with Falco (custom rule C) . . . . . 64

9.7 Scenario 7: attack tools (custom rule D) . . . . . 64

**10 Alert quality, false positives and tuning** **64**

10.1 Baseline . . . . . 64

10.2 Controlled comparison . . . . . 65

10.3 Alert quality table . . . . . 65

10.4 Noise and tuning . . . . . 67

**11 Decisions and lessons learned** **68**

11.1 Decisions . . . . . 68

11.2 Lessons learned . . . . . 68

**12 Conclusion** **70**

12.1 What the work showed . . . . . 70

12.2 Limits of the project . . . . . 71

12.3 Possible extensions . . . . . 71

**13 References** **71**

# 1 Objective

This project detects suspicious runtime behavior on a Linux host by monitoring system activity with security rules. Falco, an open-source runtime security tool, is deployed in an isolated virtual machine. Its default rules are examined, custom rules are written for selected lab behaviors, and the quality of the resulting alerts is analysed, including false positives and tuning.

## 1.1 Deliverables

- Deployment and rule configuration
- At least three documented test scenarios
- Alert evidence and interpretation
- False-positive and tuning discussion

## 1.2 Constraints and safety

- All tests run in an isolated virtual machine.
- Only systems owned by the author are tested.
- Commands, configurations, evidence, results, and lessons learned are documented in this report.

# 2 Lab environment

Falco monitors the kernel of the machine it runs on. The lab therefore had to provide a real Linux kernel that could be tested safely and restored quickly. This section describes the environment and gives the reason for each choice.

## 2.1 Environment summary

## 2.2 Why a virtual machine

The assignment requires an isolated lab. A virtual machine keeps every test away from the host computer: if a test script misbehaves or a rule change breaks the system, only the VM is affected. It also gives Falco a complete Linux kernel under the author's control, and it supports snapshots, which allow the lab to return to a known good state in seconds.

| Item | Value | Reason |
|---|---|---|
| Hypervisor | Oracle VirtualBox | Free, runs on the Windows host, and supports snapshots. A full VM gives Falco a real Linux kernel. |
| Guest OS | Ubuntu 24.04.5 LTS Desktop | Distribution preferred by the course instructor; long-term support release. |
| Hostname | `falco-lab` | Appears in every Falco alert, which makes the evidence clear. |
| Kernel | `7.0.0-34-generic` | Installed by the system updates. The kernel version matters because the Falco driver runs inside it. |
| CPU and RAM | 4 CPUs, 7000 MB assigned | Enough to run the desktop, Falco, containers, and test scripts at the same time. |
| Disk | 40 GB, dynamically allocated | The file grows only as space is used; room for Docker images and logs. |
| Network | NAT | The VM can download packages, but other devices cannot connect to it. |
| Falco | 0.45.0 (libs 0.26.0, driver API 11.0.0) | Version chosen after a compatibility check; see the compatibility analysis. |
| Driver | Modern eBPF | Needs no kernel module build and adapts to the kernel through BTF. |
| Service | `falco-modern-bpf` | systemd service that runs Falco in the background. |

**Table 1:** Lab environment and the reason for each choice

### 2.3 Why Ubuntu 24.04 LTS

Ubuntu was the distribution preferred by the course instructor. The 24.04 LTS release (long-term support) was chosen over the newer 26.04.1 LTS, which the download page offered, because it is the more established release for Falco and its surrounding tools (Docker, VirtualBox Guest Additions). This lowers the risk of compatibility problems, so that project time goes to rule analysis. The Desktop edition was used so that several terminals can be shown side by side and screenshots are easy to capture.

### 2.4 Network mode

In NAT mode the VM sits behind VirtualBox's virtual router. It can reach the internet, which is needed to install packages, but other devices on the network cannot connect to it. This keeps the lab isolated while still allowing installation.

### 2.5 Snapshots

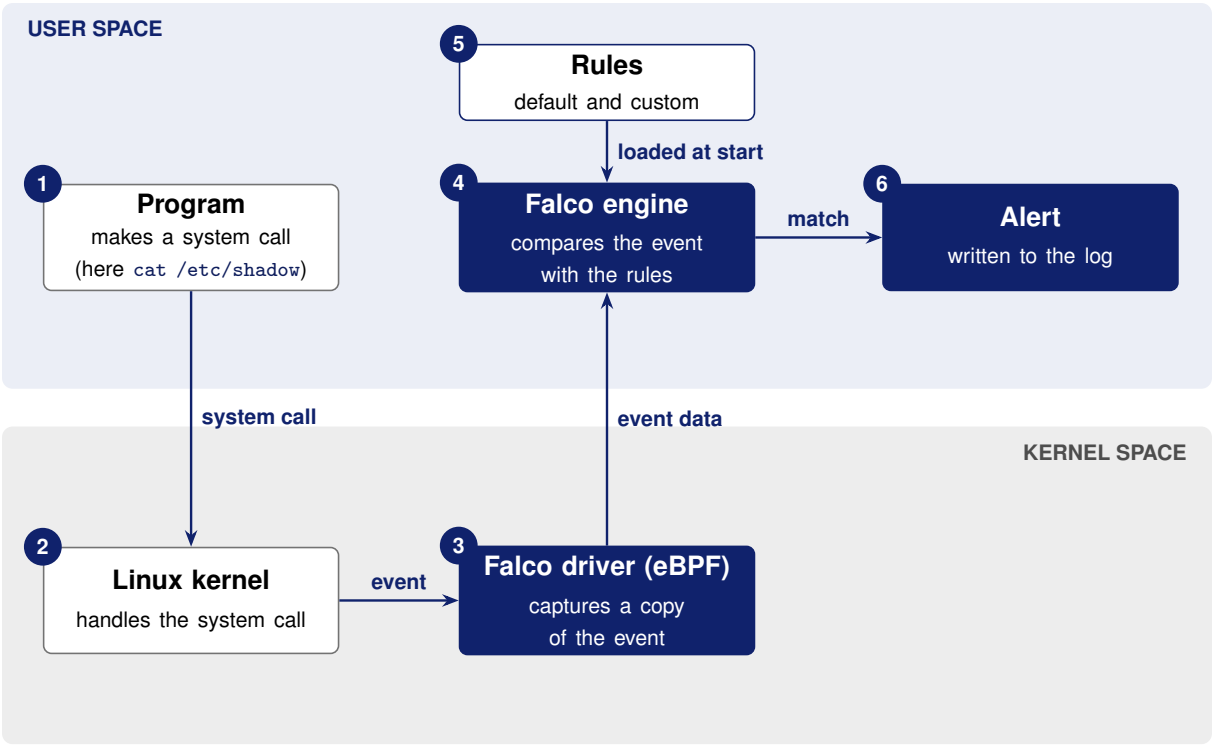
A snapshot saves the complete state of the VM. The snapshot `01-before-falco` was taken after the system was prepared and before Falco was installed, so the lab can return to a clean system if a later change fails.

## 3 How Falco works

Falco is a host-based, rule-driven runtime intrusion detection system. It does not scan files for known malware signatures, as an antivirus does, and it does not inspect network packets, as Snort or Suricata do. Instead it watches what programs do on the machine, by observing their system calls, and compares that behaviour with a set of rules. When an action matches a rule, Falco raises an alert. Falco only detects and reports; it never blocks the action.

### 3.1 Flow of one event

Every action a program takes on a Linux system passes through the kernel. Falco is built around this fact. Figure 1 follows one event from the moment a user runs a command until an alert is written. The example is the command `cat /etc/shadow`, which reads the file that stores password hashes.




*The kernel does not wait for Falco: the driver captures a copy of the event, so nothing is blocked.*

**Figure 1:** Path of one system call from a program to a Falco alert, in six steps. Dark blue boxes are Falco components; white boxes belong to the standard Linux system. The kernel and the driver run in kernel space, and the other boxes in user space.

The six steps in the figure are:

1. **Program.** The user types a command. The program that runs (here `cat`) cannot read a file by itself, because only the kernel is allowed to touch files, processes, and permissions. The program must ask the kernel to do it.
2. **Linux kernel.** The request is a *system call*; for opening a file it is `openat`. The kernel checks the permissions, performs the action, and returns the result to the program. This part of the flow is the normal operation of Linux, and it is identical with or without Falco.
3. **Falco driver.** A small program loaded into the kernel (the modern eBPF driver) is triggered on every system call. It captures a copy of the details of the call (who ran it, which process, which file, when) and sends them up to Falco in user space. It does not change or block the call.
4. **Falco engine.** Falco runs as an ordinary program outside the kernel. It receives the events from the driver and processes them.
5. **Rules.** The default and custom rules are loaded into Falco when it starts. The engine compares each event with them. Most events match no rule and are silently dropped. In the example, the event matches a rule about reading a sensitive file.
6. **Alert.** When a rule matches, Falco builds a message from the rule's output template, adds the rule's priority, and writes it to the configured output. In this lab the alert is read from the system log of the `falco-modern-bpf` service.

Because Falco works on a copy of the event, the original action is never delayed. The kernel does not wait for Falco, as the note under the figure says, and this is why Falco cannot block anything.

## 3.2 System calls and event details

A system call is the only way a program can use a resource that the kernel controls: opening a file, starting a process, changing a permission, or opening a network connection. A program cannot avoid system calls, so watching them gives a complete view of what the program does on the host.

For every system call, the driver collects a set of details. The main ones are the name of the call, the file involved, the user, the process name and full command line, the process identifier and its parent process, whether the process runs inside a container, and the time. These details are the *fields* that rule conditions are written with. For example, a condition may test `evt.type` (the call), `fd.name` (the file), `user.name` (the user), or `proc.name` (the process). The quality of a rule therefore depends directly on which of these fields it checks.

## 3.3 The driver

The rules engine runs outside the kernel and cannot see inside it, so something must run inside the kernel to observe system calls and pass the details out. This component is the driver. Falco supports three kinds of driver, summarised in Table 2.

| Driver | How it works | Main drawback |
|---|---|---|
| Kernel module | Code loaded into the kernel with full privileges. | A fault in it can crash the whole machine; it must match the exact kernel version. |
| Legacy eBPF probe | A small program accepted by the kernel after a safety check. | Typically has to be built or downloaded for each kernel version. |
| Modern eBPF | A checked program embedded in Falco itself that adapts to the running kernel. | Needs a reasonably recent kernel that provides BTF. |

**Table 2:** The three Falco driver types

**What eBPF is.** eBPF lets a small program be loaded into the kernel and attached to an event, here a system call. Before the kernel accepts the program, a component called the *verifier* inspects it. It rejects any program that could loop forever, read memory it should not, or crash the kernel. A program that fails the check is never loaded, which is why eBPF is considered safer than a kernel module.

**Why BTF matters.** Different kernels arrange their internal data differently, and the driver must know where to find, for example, the file name inside a system call. BTF is the kernel's own description of its internal layout. With it, the modern eBPF driver adjusts itself to the running kernel when it loads, an approach known as CO-RE (compile once, run everywhere). The file `/sys/kernel/btf/vmlinux` is the BTF description, and its presence was verified on the lab VM before Falco was installed.

**Driver choice in this project.** The modern eBPF driver was used. It is the safest of the three options, it needs no kernel-specific build, and it ran on the lab's new kernel (7.0.0-34) without a driver download. Two separate questions had to be checked: whether the driver can adapt to the kernel (BTF present), and whether the driver code itself is correct on that kernel. For the second question, Falco 0.44.x is reported to crash on 7.0 kernels (issue #3955 in the `falcosecurity/falco` repository), and the fix ships in Falco 0.45.0, so that version was installed. This is covered in the compatibility analysis.

## 3.4 The rules engine

The rules engine applies every loaded rule to every event it receives. A rule consists of a condition, which is a test over the event fields; an output, which is the alert text with fields filled in; and a priority, which states the severity. If the condition is true for an event, the rule fires and an alert is produced. If not, the event is dropped. The structure of a rule, and how default rules are built from lists and macros, is examined in detail in the rules review later in this report.

## 3.5 IDS versus IPS

The two kinds of security system differ in where they sit relative to the action they watch.

- An **intrusion detection system (IDS)** observes a copy of the activity on the side. The action is not delayed, and the system reports afterwards that something suspicious happened. Falco is an IDS.
- An **intrusion prevention system (IPS)** sits in the path of the action. The action is paused, the system decides whether to allow or block it, and only then does the action continue or get refused. This can stop an attack before it happens, but it adds delay to every checked action, and a fault in the IPS can disturb normal work.

This design makes Falco safe and cheap to run, but it has a limitation that is relevant for the later analysis: the alert arrives after the event. In the example above, the password file has already been read by the time the alert appears. Falco tells the administrator quickly what happened; it does not prevent it.

# 4 Deployment

This section documents how the lab was prepared and how Falco was installed and verified. Every command is shown exactly as it was typed, followed by an explanation of what it does and what result it produced. The commands were run inside the Ubuntu virtual machine `falco-lab`.

## 4.1 System preparation

**Goal.** Falco works by observing the kernel, so the lab starts from a fully updated system. The VirtualBox Guest Additions are installed so that the virtual machine supports a shared clipboard and automatic screen resizing.

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y virtualbox-guest-x11
sudo reboot
```

- `sudo` runs a command with administrator rights, which installing software requires.
- `apt` is Ubuntu's package manager. `apt update` only downloads the latest list of available packages. It installs nothing.
- `&&` runs the second command only if the first one succeeded.
- `apt full-upgrade` installs every available update, including updates that require other packages to be added or removed, such as kernel updates. The option `-y` answers the confirmation question with yes.
- `virtualbox-guest-x11` is the Guest Additions package for the graphical desktop.
- `reboot` restarts the machine so that the updated kernel is loaded.

**Result.** All commands completed without errors. After the reboot the system ran kernel `7.0.0-34-generic`.

## 4.2 Environment readiness check

**Goal.** Before installing Falco, confirm that the system meets its requirements. The most important one is the kernel: the Falco driver runs inside it, so its version and features decide whether Falco can work.

```
lsb_release -a
uname -r
ls -l /sys/kernel/btf/vmlinux
nproc && free -h
dpkg -l | grep linux-image
apt-cache search '^linux-image-6\.8\.0-[0-9]+-generic$' | tail -3
df -h / | tail -1
```

| Check | Why it matters | Result |
|---|---|---|
| Operating system | Confirms the distribution and release. | Ubuntu 24.04.5 LTS (noble) |
| Kernel version | The driver runs in the kernel, so the version decides compatibility. | `7.0.0-34-generic` |
| BTF file | The modern eBPF driver needs it to adapt to the kernel. | Present, about 7 MB |
| CPU and memory | Confirms the resources given to the virtual machine. | 4 CPUs, 6.6 GiB RAM |
| Installed kernels | Shows whether an older kernel exists as a fallback. | Only 7.0.0-34 |
| Fallback kernel | Shows whether a 6.8 kernel could be installed if needed. | 6.8.0-71 available |
| Free disk space | Room for container images and logs. | 27 GB free of 40 GB |

**Table 3:** Readiness checks and their results

How the commands work: `uname -r` prints the kernel release. `ls -l /sys/kernel/btf/vmlinux` lists the BTF file, which the kernel creates only if it provides BTF. `nproc` and `free -h` show the CPUs and the memory. The pipe symbol `|` sends the output of one command to the next, so `dpkg -l | grep linux-image` keeps only the installed kernels. `apt-cache search` with a regular expression (a pattern of names) finds the 6.8.0 kernel image packages and nothing else, and `tail -3` keeps the last three matches. `df -h /` shows the disk usage of the root filesystem.

**Interpretation.** The BTF file exists, so the modern eBPF driver has what it needs. The kernel is very new, which is why the Falco version was checked before installation (see the compatibility analysis). Only one kernel was installed, but a 6.8 kernel is available from the repository as a fallback.

## 4.3 Clipboard fix

**Problem.** Copy and paste between Windows and the virtual machine did not work. Ubuntu 24.04 uses the Wayland display system by default, and the VirtualBox clipboard helper was not running.

```
pgrep -a VBoxClient
dpkg -l | grep virtualbox-guest | cut -c1-60
lsmod | grep vbox
VBoxClient --clipboard
```

The empty output of `pgrep` showed that the helper was not running. The `dpkg` and `lsmod` commands showed that the Guest Additions packages were installed and that their kernel modules (`vboxguest` and `vboxsf`) were loaded, so only the helper was missing. `VBoxClient -clipboard` started it by hand, and its log reported that a Wayland environment was detected.

**Result.** Copy and paste worked after starting the helper. The helper has to be started again after a new login unless it is added to the autostart list.

## 4.4 Connectivity check

**Goal.** Falco is downloaded from the project's own package repository, so the virtual machine must be able to reach it.

```
ping -c 2 download.falco.org
sudo apt install -y curl
curl -sI https://download.falco.org | head -1
```

`ping` resolved the name but received no replies, which is common for servers that do not answer ping. `curl`, a download tool that was added with `apt`, was then used with `-sI` to request only the response headers and keep the first line.

**Result.** The first line was `HTTP/2 307`, a redirect. The server answered, so the connection to the Falco repository works.

## 4.5 Falco repository

**Goal.** Falco is not installed from Ubuntu's default sources, so Ubuntu must be told where the Falco packages are and which signing key to trust.

```
sudo apt install -y gnupg
curl -fsSL https://falco.org/repo/falcosecurity-packages.asc | sudo gpg --dearmor -o /usr/share/
    keyrings/falco-archive-keyring.gpg
ls -l /usr/share/keyrings/falco-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/falco-archive-keyring.gpg] https://download.falco.org/
    packages/deb stable main" | sudo tee /etc/apt/sources.list.d/falcosecurity.list
sudo apt update
apt-cache policy falco
```

- `curl -fsSL ... | sudo gpg -dearmor -o ...` downloads Falco's public signing key and converts it to the binary form that apt reads. `sudo` is needed because the keyring folder belongs to the administrator.
- The `echo` line writes one repository entry. `signed-by` limits trust to the downloaded key for this repository only, so packages that are not signed with that key are rejected. `| sudo tee` writes the line into a protected file, because a plain `>` redirection would run without the rights that `sudo` grants.
- `apt update` reloads the package lists, now including Falco's, and `apt-cache policy falco` shows the versions available. `Candidate` is the version apt would install.

**Result.** The key file was created (1189 bytes). The candidate version was Falco **0.45.0**, the first version with the fix for the kernel 7.0 problem described in the compatibility analysis. The repository also lists many older versions. They were not used.

## 4.6 Installation and verification

**Goal.** Install Falco and prove that it is running correctly on the lab's kernel. A snapshot (`01-before-falco`) was taken just before the installation, as a rollback point.

```
sudo apt install -y falco
falco --version
systemctl list-units 'falco*' --no-pager
systemctl show falco-modern-bpf -p NRestarts,ActiveState,SubState
```

- `apt install -y falco` installs the candidate version, 0.45.0, and sets Falco up as a background service.
- `falco -version` prints the version information.
- `systemctl` controls services. `list-units 'falco*'` lists the services whose name begins with `falco`.
- `systemctl show` prints selected properties of a service. `NRestarts` counts how often systemd had to restart it, which would reveal a crash.

| Item | Value reported by Falco |
|---|---|
| Falco version | 0.45.0 |
| Libraries (libs) | 0.26.0 |
| Rules engine | 0.65.0 |
| Plugin API | 3.12.0 |
| Driver API version | 11.0.0 |
| Default driver | 11.0.0+driver |
| Kernel reported | Linux 7.0.0-34-generic |

**Table 4:** Output of `falco -version`

Falco also reported that three configuration files passed schema validation. They are examined in the configuration section.

- `/etc/falco/falco.yaml`
- `/etc/falco/config.d/engine-kind-falcoctl.yaml`
- `/etc/falco/config.d/falco.container_plugin.yaml`

Two services were running:

- `falco-modern-bpf.service` is Falco itself, using the modern eBPF driver. This is the service used for status checks and logs.
- `falcoctl-artifact-follow.service` is a helper that follows updates of Falco's rule files from the project. Automatic rule updates could change alert behaviour between test runs, but this service was not reviewed in this project (see the configuration section).

**Result.** The service showed `ActiveState=active`, `SubState=running`, and `NRestarts=0`. Falco therefore started on kernel 7.0.0-34 and had not crashed or restarted. This check was made only a few minutes after installation, so it is a first indication. A longer stability test was not run; this is discussed in the compatibility analysis.

# 5 Compatibility analysis

Falco's driver runs inside the Linux kernel. A combination of Falco and kernel that has not been tested together can therefore fail in ways that do not show up during installation. This section records how the compatibility between Falco and the lab's kernel was checked.

## 5.1 The situation

After the system update, the lab virtual machine ran kernel `7.0.0-34-generic`. The standard kernel of Ubuntu 24.04 belongs to the 6.8 series. The 7.0 kernel comes from Ubuntu's hardware enablement updates, which move a long-term release to newer kernels over time. A very new kernel is a risk for a kernel-level tool such as Falco, because the driver depends on the internal behaviour of the kernel it runs in.

## 5.2 The reported problem

The Falco issue tracker contains a report (issue #3955 in the `falcosecurity/falco` repository) that Falco 0.44.x with the modern eBPF driver can stop working on Linux 7.0 kernels. According to the report, Falco starts normally and fails later with errors about events that cannot be parsed, and users saw it on Ubuntu 24.04 systems running 7.0 kernels. The problem is described as a timing fault (a race condition) in the driver. The fix is included in Falco 0.45.0, which uses libs 0.26.0 and driver API 11.0.0. The release notes of 0.45.0 also mention a driver fix for a modern eBPF race on preemptible kernels. The version string of the lab kernel contains `PREEMPT_DYNAMIC`, a kernel built with selectable preemption, so the report was relevant to this lab.

## 5.3 Checks performed before relying on Falco

Instead of installing Falco and waiting for a failure, the following checks were made.

| Question | How it was checked | Result |
|---|---|---|
| Does the kernel provide BTF? | Existence of `/sys/kernel/btf/vmlinux` | Yes, the file is present |
| Which Falco version does the repository offer? | `apt-cache policy falco` | 0.45.0, the version with the fix |
| Do the installed components match the fix? | `falco -version` | libs 0.26.0, driver API 11.0.0 |
| Does the service run without restarts? | `systemctl show falco-modern-bpf -p NRestarts` | 0 restarts shortly after installation |
| Is there a fallback kernel? | `apt-cache search` for 6.8 kernels | 6.8.0-71 is available |

**Table 5:** Compatibility checks between Falco and kernel 7.0.0-34

### 5.4 Fallback plan

If Falco had failed on kernel 7.0, the plan was to install the 6.8.0-71 kernel and its module packages with `apt`, choose it in the boot menu (GRUB), and repeat the checks. The snapshot `01-before-falco` allows the installation itself to be rolled back. The fallback was not needed during the installation checks.

### 5.5 Stability

No dedicated long-running stability test was performed. The conclusion that Falco 0.45.0 works on kernel 7.0.0-34 rests on the checks above (BTF present, the version that contains the fix, matching library and driver versions) and on the use of the service during the whole project: it was running for every test, it was restarted by hand many times, it started again by itself at each boot, and no failure of the kind reported for Falco 0.44.x was observed. The only measurement of restarts, `NRestarts=0` a few minutes after the installation, is a first indication. A run of at least 30 minutes under activity, with a count of restarts, is a possible extension.

## 6 Falco configuration

### 6.1 Rules files and loading order

Before writing any rule, the lab had to show which rules files Falco loads and in which order, because a rule placed in a file that Falco ignores has no effect.

```
ls -l /etc/falco/ && grep -n -A6 "^rules_files" /etc/falco/falco.yaml
```

- `ls -l /etc/falco/` lists the configuration folder with sizes and dates.
- `grep -n -A6 "^rules_files"` finds the setting that lists the rules files in `falco.yaml`, and prints 6 lines after it.

```
rules_files:
  - /etc/falco/falco_rules.yaml
  - /etc/falco/falco_rules.local.yaml
  - /etc/falco/rules.d
```

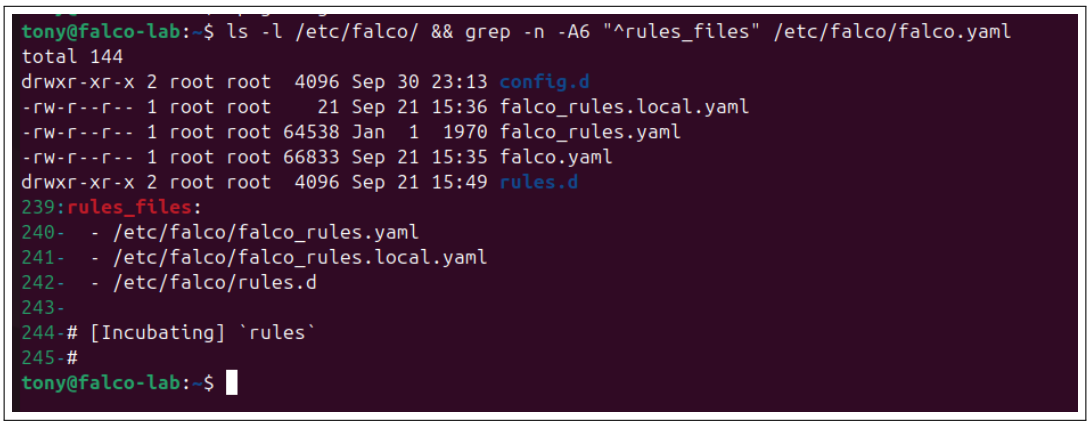



**Figure 2:** Rules files loaded by Falco and their order

Falco loads the default rules first, then the local file, then the folder `rules.d`. A definition in a later file can override one from an earlier file, so an exception written in the local file takes effect after the default rule is read. The local file `falco_rules.local.yaml` already existed and was 21 bytes long. Its only content was the comment `# Your custom rules!`, so it was safe to extend. The folder `rules.d` is a second place where custom rules can be stored.

### 6.2 Observations at startup

The service log of the restart showed three details that are noted here and reviewed later:

- The buffer in which the driver passes system call events to Falco is 8 MB (8388608 bytes).
- Falco starts a small health web server that listens on `0.0.0.0:8765`, which means all network interfaces. The virtual machine uses NAT, so other devices cannot reach it.
- The container plugin enables several container runtime sockets (Docker, containerd, CRI-O, Podman and others), so Falco is prepared to recognise events from containers.

### 6.3 Automatic reload

The configuration file `falco.yaml` contains the setting `watch_config_files: true`, marked as stable. With it, Falco monitors its configuration and rules files and reloads itself when they change. This was observed in the log of the service and is described with the first custom rule (Section 8.1): a rule saved in the local file is live at once, without a restart.

### 6.4 Settings not reviewed

The output settings, the priority threshold, and the update helper `falcoctl` were not reviewed in this project. The alerts were read from the log of the service, and the effect of the helper on the rules folder was not measured.

## 7 Default rules review

The assignment asks for an examination of Falco's default rules. Instead of reading the rules file from the top, this review starts from a real alert produced by the lab and works back to the rule that caused it. This shows how a rule is built, and it also exposes the noise that tuning has to deal with.

### 7.1 Method used for each rule

The same method is applied to every default rule examined in this report. Table 6 shows it, with what was done for the first rule. The following subsections describe each step in detail.

Steps 1 to 4 are applied to every rule and scenario. Steps 5 to 8 are applied to the rules that are selected for tuning.

| # | Step | What was done for the first rule | Result |
|---|---|---|---|
| 1 | Get the alert | Read the first alert in the Falco log with `journalctl`. | The login manager read a login file, `/etc/pam.d/gdm-password`. |
| 2 | Find the rule | Searched the rules folder for a part of the alert text with `grep`. | The rule *Read sensitive file untrusted*, in the default rules file. |
| 3 | Read the rule | Printed the rule with `sed` and read the macros it uses. | It fires when a sensitive file is read by a program that is not on the trusted list. |
| 4 | Measure the noise | Counted the alerts per file and per program. | 51 alerts, all from one program. |
| 5 | Verify the program | Checked it against its package with `dpkg -S` and `dpkg -V`. | Genuine package file, so the alerts are false positives. |
| 6 | Tune | Wrote an exception for the exact program path in the local rules file. | The default file stays unchanged. |
| 7 | Validate and restart | Validated both files with `falco -V`, restarted Falco, and checked the loaded files. | Both files `Ok` and loaded. |
| 8 | Measure again and control | Counted after one boot and one login, then read `/etc/shadow` with `cat`. | 27 alerts before, 0 after; the control still raised an alert. |

**Table 6:** Method applied to each rule, with the first rule as the example

## 7.2 Where the default rules are

Falco's default rules are stored in `/etc/falco/falco_rules.yaml`. This file is not edited: a package update can replace it, and changes inside it would be hard to tell apart from the defaults. Custom rules are written in a separate file, `falco_rules.local.yaml`. How Falco loads that file is checked in the configuration section.

## 7.3 Starting from a real alert

Falco raised alerts without any action from the author. They appeared in the service log shortly after the installation and again after the reboot. The first one was read in full with this command:

```
sudo journalctl -u falco-modern-bpf -o cat --no-pager | grep -m 1 Sensitive
```

- `journalctl` reads the system log, and `-u falco-modern-bpf` limits it to the Falco service.
- `-o cat` prints only the message text, without the date and host prefix, which makes the alert easier to read.
- `grep -m 1 Sensitive` keeps the first line that contains the word `Sensitive` and then stops.

This is the first matching alert in the log:

```
23:14:31.967774636: Warning Sensitive file opened for reading by non-trusted program | file=/etc
    /pam.d/gdm-password gparent=gdm3 ggparent=systemd gggparent=<NA> evt_type=openat user=root
    user_uid=0 user_loginuid=1000 process=gdm-session-wor proc_exepath=/usr/libexec/gdm-session-
    worker parent=gdm-session-wor command=gdm-session-wor terminal=0 container_id=host
    container_name=host container_image_repository= container_image_tag= k8s_pod_name=<NA>
    k8s_ns_name=<NA>
```

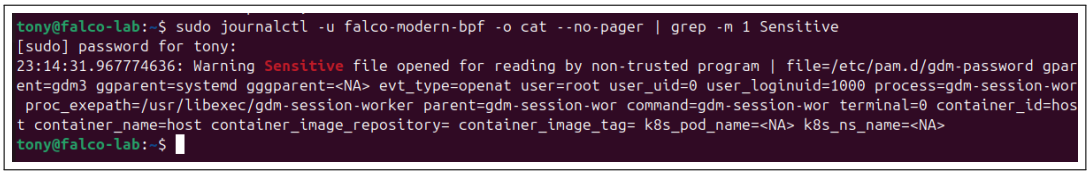



**Figure 3:** First alert read from the Falco service log

| Field | Value | Meaning |
|---|---|---|
| file | /etc/pam.d/gdm-password | The file that was opened: the login configuration of the graphical login screen. |
| evt_type | openat | The system call used to open the file. |
| user | root | The program ran with administrator rights. |
| user_loginuid | 1000 | The account that originally logged in, which is the lab user. |
| process | gdm-session-wor | Program name, cut at 15 characters (a Linux limit on process names). |
| proc_exepath | /usr/libexec/ gdm-session-worker | Full path of the program, located in a system folder. |
| parent, gparent, ggparent | gdm-session-wor, gdm3, systemd | The chain that started it: systemd started the login manager, which started the session worker. |
| terminal | 0 | No terminal is attached, so a background program ran it and not a shell. |
| container_id | host | The event happened on the machine and not inside a container. |

**Table 7:** Fields of the first alert

The alert reports that the login manager of the desktop read one of its own login configuration files while a session was being started. Nothing in the fields suggests an attack, but the alert alone does not prove it, so the rule behind it is examined next.

## 7.4 Finding the rule behind the alert

The alert text comes from the output template of a rule. Searching the rules folder for a distinctive part of the message finds that rule.

```
sudo grep -rn "non-trusted program" /etc/falco/
```

- `grep` searches for text; `-r` searches every file in the folder and `-n` adds the line number of each match.
- `sudo` is needed because some files in the folder are protected.

The match was on line 431 of `/etc/falco/falco_rules.yaml`, the `output:` line of the rule. The complete rule was then printed with `sed`, which prints only a chosen range of lines:

```
sudo sed -n '397,433p' /etc/falco/falco_rules.yaml
```

`-n` switches off the default printing of every line, and `'397,433p'` prints lines 397 to 433 only.

## 7.5 Anatomy of the rule *Read sensitive file untrusted*

```
- rule: Read sensitive file untrusted
  desc: >
    An attempt to read any sensitive file (e.g. files containing user/password/authentication
    information). Exceptions are made for known trusted programs. Can be customized as needed.
    In modern containerized cloud infrastructures, accessing traditional Linux sensitive files
    might be less relevant, yet it remains valuable for baseline detections. While we provide
    additional
    rules for SSH or cloud vendor-specific credentials, you can significantly enhance your
    security
    program by crafting custom rules for critical application credentials unique to your
    environment.
  condition: >
    open_read
    and sensitive_files
    and proc_name_exists
    and not proc.name in (user_mgmt_binaries, userexec_binaries, package_mgmt_binaries,
     cron_binaries, read_sensitive_file_binaries, shell_binaries, hids_binaries,
     vpn_binaries, mail_config_binaries, nomachine_binaries, sshkit_script_binaries,
     in.proftpd, mandb, salt-call, salt-minion, postgres_mgmt_binaries,
     google_oslogin_
     )
    and not cmp_cp_by_passwd
    and not ansible_running_python
    and not run_by_qualys
    and not run_by_chef
    and not run_by_google_accounts_daemon
    and not user_read_sensitive_file_conditions
    and not mandb_postinst
    and not perl_running_plesk
    and not perl_running_updmap
    and not veritas_driver_script
    and not perl_running_centrifydc
    and not runuser_reading_pam
    and not linux_bench_reading_etc_shadow
    and not user_known_read_sensitive_files_activities
    and not user_read_sensitive_file_containers
    and not systemd_sensitive_file_readers
  output: Sensitive file opened for reading by non-trusted program | file=%fd.name gparent=%proc
    .aname[2] ggparent=%proc.aname[3] gggparent=%proc.aname[4] evt_type=%evt.type user=%user.
    name user_uid=%user.uid user_loginuid=%user.loginuid process=%proc.name proc_exepath=%proc.
    exepath parent=%proc.pname command=%proc.cmdline terminal=%proc.tty
  priority: WARNING
  tags: [maturity_stable, host, container, filesystem, mitre_credential_access, T1555]
```

| Part | Content and meaning |
|---|---|
| `rule` | The name of the rule. It is not the same as the alert message. |
| `desc` | A plain-language purpose: detect an attempt to read a sensitive file, with exceptions for known trusted programs. It also states that the rule can be customized. |
| `condition` | The test applied to every event. All parts joined by `and` must be true for the rule to fire. |
| `output` | The alert text. Each `%field` is replaced with the value from the event. |
| `priority` | The severity of the alert: `WARNING`. |
| `tags` | Labels for grouping and filtering, including a reference to the MITRE ATT&CK framework. |

**Table 8:** Parts of the rule

### Reading the condition

The condition is built from three kinds of names:

- **Fields** contain a dot, such as `proc.name`. They are the event details collected by the driver.
- **Macros** are reusable pieces of condition, such as `open_read` or `sensitive_files`. They are defined elsewhere in the file.
- **Lists** are named sets of values, such as `user_mgmt_binaries`, used after `in (...)`.

The condition says: fire when a file is opened for reading, the file is sensitive, and the process has a name, *and* the program is not in any of the trusted groups, *and* none of the listed exceptions applies. It is an allow-list: every read of a sensitive file is reported unless the program is known to be legitimate. Apart from the exception macros, which were not all read, the condition does not test the user name. The decision rests on *which program* touched the file.

### Priority and tags

The priority `WARNING` marks the alert as worth attention but not critical. The tags include `maturity_stable`, which is the Falco project's own label for a mature rule, `host` and `container`, which show that the rule applies in both places, and `filesystem`. The tags `mitre_credential_access` and `T1555` link the rule to the MITRE ATT&CK catalogue of attacker techniques.

In the MITRE ATT&CK catalogue, `T1555` is called *Credentials from Password Stores*. For reading `/etc/shadow` on Linux, the sub-technique `T1003.008`, *OS Credential Dumping: /etc/passwd and /etc/shadow*, describes the behaviour more closely, so the tag of the default rule is broader than the behaviour that was tested.

## 7.6 The definitions behind the condition

Two names in the condition decide why the alert fired and where it can be tuned. Their definitions were found with:

```
sudo grep -n -A8 -E "^- (macro|list): (sensitive_files|
    user_known_read_sensitive_files_activities)$" /etc/falco/falco_rules.yaml
```

- `-A8` prints 8 lines after each match, enough to show a whole definition.
- `-E` enables a pattern with alternatives.
- The pattern matches a line that starts with `- macro:` or `- list:` followed by one of the two names.

```
- macro: sensitive_files
  condition: >
    (fd.name in (sensitive_file_names) or
      fd.directory in (/etc/sudoers.d, /etc/pam.d))

- macro: user_known_read_sensitive_files_activities
  condition: (never_true)
```

**Why the alert fired.** A file counts as sensitive if its full path is in the list `sensitive_file_names`, or if it lies in `/etc/sudoers.d` or `/etc/pam.d`. The file in the first alert, `/etc/pam.d/gdm-password`, is in the second folder. The rule is broad on purpose, because PAM files control how logins are checked. The cost is that any program that reads them during a normal login also matches.

**Where the rule can be tuned.** The macro `user_known_read_sensitive_files_activities` is defined as `never_true`, a condition that is always false. The rule contains `and not` followed by this macro, so today it changes nothing. Its name and its definition indicate that it is a placeholder where local exceptions can be added without editing the default file. The rule that follows in the file, *Read sensitive file trusted after startup* (line 341), is a related rule and is reviewed later.

## 7.7 Baseline before tuning

Before changing anything, the amount of noise produced by this rule was measured, so that tuning can later be compared with a starting point.

```
sudo journalctl -u falco-modern-bpf -o cat --no-pager | grep "non-trusted program" | grep -o "
    file=[^ ]*" | sort | uniq -c | sort -rn
```

- `grep "non-trusted program"` keeps only the alerts of this rule.
- `grep -o` prints only the part of each line that matches the pattern. Here the pattern is `file=` followed by every character up to the next space, so only the file name of each alert is printed.
- `sort` puts identical lines next to each other, `uniq -c` merges them and adds a count, and `sort -rn` orders the result with the highest count first.

| File | Alerts |
|---|---|
| /etc/shadow | 11 |
| /etc/pam.d/common-session | 8 |
| /etc/pam.d/common-password | 8 |
| /etc/pam.d/common-auth | 8 |
| /etc/pam.d/common-account | 8 |
| /etc/pam.d/other | 4 |
| /etc/pam.d/gdm-password | 4 |
| **Total** | **51** |

**Table 9:** Alerts of the rule per file, before tuning (since the installation)

The count covers the whole log since Falco was installed, so it mixes the installation, the first boot, and the second boot. It is a first measurement. A controlled comparison with the same activity before and after tuning is made in the alert quality section.

The file names vary, so the program behind each alert was checked as well:

```
sudo journalctl -u falco-modern-bpf -o cat --no-pager | grep "non-trusted program" | grep -oE "(
    file|process|proc_exepath)=[^ ]*" | paste - - - | sort | uniq -c | sort -rn
```

The extra step `paste - - -` joins every three extracted values into one row, so that each alert becomes one line with the file, the process name, and the program path. All 51 alerts have the same program:

```
process=gdm-session-wor
proc_exepath=/usr/libexec/gdm-session-worker
```

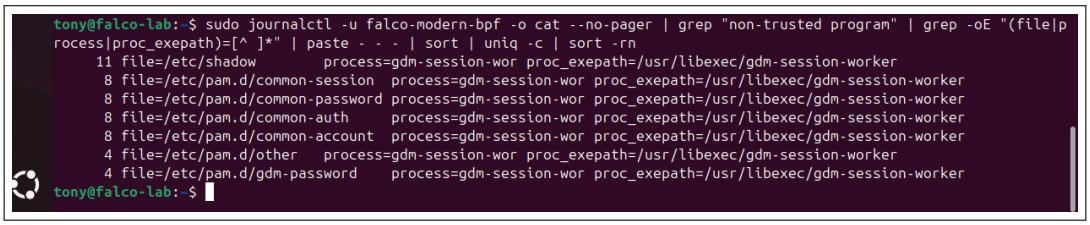



**Figure 4:** Alert count per file and program before tuning

Grouping by file suggested several sources, but grouping by program showed a single one. The GNOME login manager is the source of all the alerts of this rule.

## 7.8 Verifying the program

Before an exception is written, the flagged program has to be checked. An exception for a program that is not what it claims to be would hide an attack.

```
dpkg -S /usr/libexec/gdm-session-worker
dpkg -V gdm3
```

- `dpkg` is Ubuntu's package tool. It keeps a record of every file that each package installed, together with a checksum of its contents.
- `dpkg -S` (search) asks which package owns a file.
- `dpkg -V` (verify) compares every installed file of a package with the checksum stored at installation time. No output means that no file differs.

The first command printed:

```
gdm3: /usr/libexec/gdm-session-worker
```

The second command printed nothing. Because the first command had shown that the package is installed, the empty output means that `dpkg` checked the files and found no differences. This is different from the empty result of a search that matches nothing.

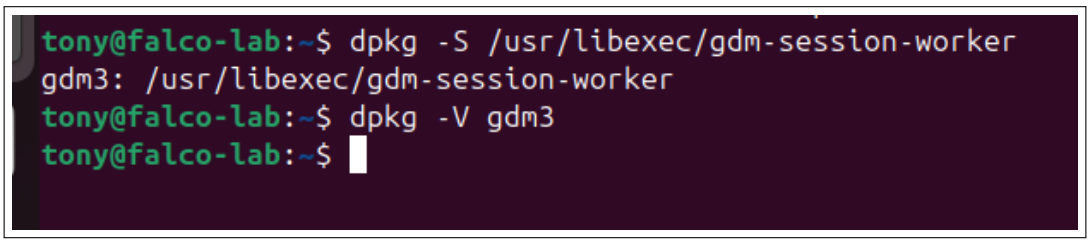



**Figure 5:** Ownership and verification of the login manager program

Both checks compare the file with records stored on the same machine. They are a reasonable check for this lab, but they would not protect against an attacker who already controls the system and can change both the file and the records.

## 7.9 Diagnosis

- **Why the rule fires.** The folder `/etc/pam.d` and the file `/etc/shadow` count as sensitive, and the login manager program is not in any trusted group or exception of the rule.
- **Why it is harmless.** The program is installed in the system folder `/usr/libexec`, it was started by `gdm3` and `systemd`, the files it reads are the ones a login needs, and it belongs to the `gdm3` package and matches its recorded checksums.
- **Part of the cause is the lab setup.** The Ubuntu Desktop edition runs a graphical login manager, which a server installation would not.
- **How the exception should be written.** The default rules file itself explains that helper programs report truncated names and that the program path (`proc.exepath`) is resolved by the kernel and is harder to fake. An exception should therefore use the full path and not the short process name.

The alerts of this rule that come from the login manager are therefore classified as **false positives**.

## 7.10 Tuning applied

The goal of the tuning is to remove this false positive without weakening the detection of other programs.

### Options considered

Table 10 compares the ways that were considered.

| Option | Effect | Verdict |
|---|---|---|
| Set the hook macro to `always_true` | The rule never fires for any program. | Too broad: removes the detection. |
| Remove `/etc/pam.d` from `sensitive_files` | No program is reported for PAM files any more. | Too broad: changes what counts as sensitive. |
| Fill the hook macro with the exact program path | Only that program is ignored. | Chosen. |
| Exact path plus the parent process | Even tighter. | Possible improvement, not applied. |

**Table 10:** Ways to remove the false positive

The macro `user_known_read_sensitive_files_activities` is the place that the rule provides for exceptions. Falco's documentation on controlling rules describes overriding such `user_` macros in the local configuration instead of editing the default file.

### The exception

The exception was written in the local rules file, which was opened with `sudo nano /etc/falco/falco_rules.local.yaml`:

```
# Your custom rules!

# Exception: the GNOME login manager session worker reads PAM and shadow
# files while a user logs in. Verified with dpkg -S and dpkg -V gdm3.
- macro: user_known_read_sensitive_files_activities
  condition: (proc.exepath = "/usr/libexec/gdm-session-worker")
  override:
    condition: replace
```

Table 11 explains each line.

The rule contains `and not user_known_read_sensitive_files_activities`. When the reading program is the login manager worker, the macro is true, its negation is false, and the rule stays silent. Every other program is still reported. The default rules file was not changed. The hook is also used by a second rule, *Read sensitive file trusted after startup* (lines 356 and 428

| Line | Meaning |
|---|---|
| `# Exception: ...` | Comments that record why the exception exists. |
| `- macro: user_known_read_sensitive_files_activities` | Defines a macro with the same name as the empty hook in the default file. |
| `condition: (proc.exepath = "...")` | The new meaning of the hook: true only when the full path of the program is exactly the login manager worker. |
| `override: condition: replace` | Tells Falco to discard the old definition, `(never_true)`, and use this one. |

**Table 11:** The lines of the exception

of the default file use it), so the exception applies to that rule as well. The alerts of that second rule were not counted, so its change is not measured here.

```
tony@falco-lab:~$ cat /etc/falco/falco_rules.local.yaml
# Your custom rules!

# Exception: the GNOME login manager session worker reads PAM and shadow
# files while a user logs in. Verified with dpkg -S and dpkg -V gdm3.
- macro: user_known_read_sensitive_files_activities
  condition: (proc.exepath = "/usr/libexec/gdm-session-worker")
  override:
    condition: replace
tony@falco-lab:~$
```

**Figure 6:** The exception in the local rules file

### Validation and restart

The file was checked before Falco was restarted, so that a mistake could not break the running service.

```
sudo falco -V /etc/falco/falco_rules.yaml -V /etc/falco/falco_rules.local.yaml
sudo systemctl restart falco-modern-bpf
systemctl status falco-modern-bpf --no-pager | head -12
sleep 10; systemctl status falco-modern-bpf --no-pager | head -12
sudo journalctl -u falco-modern-bpf --since "10:51" -o cat --no-pager | grep -A4 "Loading rules"
```

- `falco -V` validates a rules file and exits without starting the monitoring. Both files were passed in loading order, because the `replace` only makes sense after the default macro exists. Both were reported as `Ok`.
- `systemctl restart` stops and starts the service, which reloads all rules files, including the local one.
- The first status was read about 10 milliseconds after the restart. It showed the process as `(falco)` in brackets, 1 task, and 272 KB of memory, so Falco was still starting and the reading was not conclusive.
- After waiting 10 seconds, the status showed `active (running)` with the full command line, 16 tasks, and about 70 MB of memory.
- The log of the restart (the time `10:51` is when it happened) showed that both

`falco_rules.yaml` and `falco_rules.local.yaml` were loaded with schema validation ok.

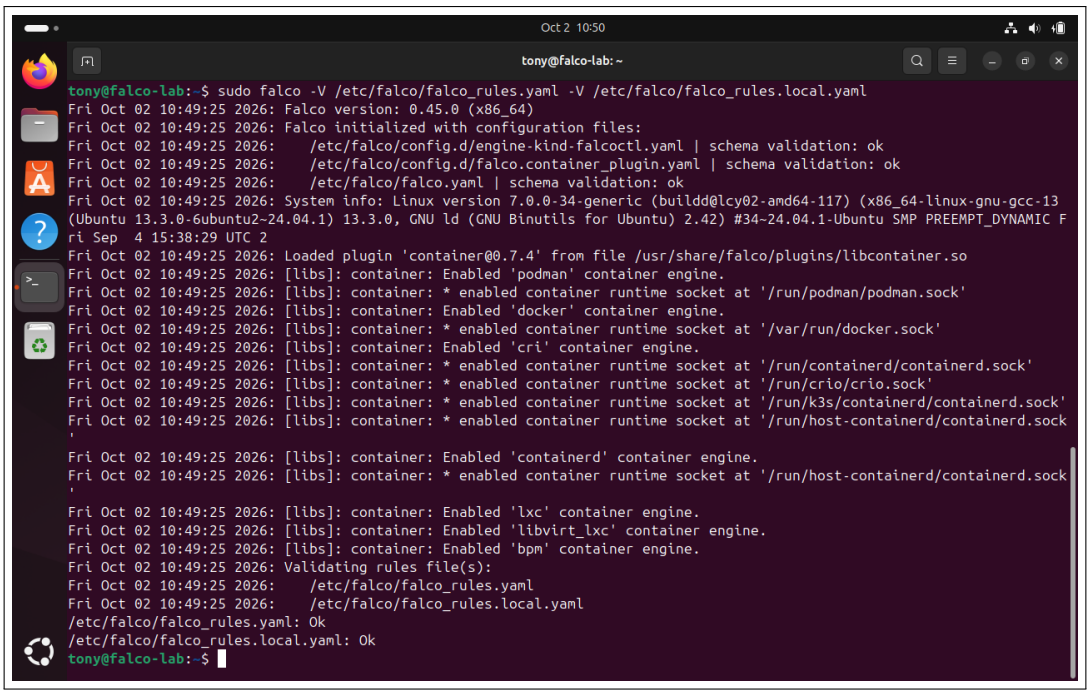



**Figure 7:** Validation of both rules files

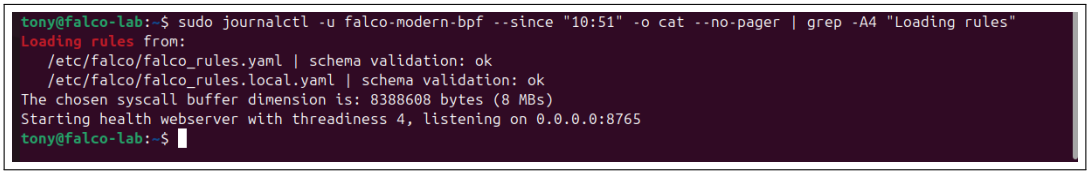



**Figure 8:** Rules files loaded after the restart

### Measuring the effect

The effect was measured with the same procedure before and after: one boot and one login, then a count of the alerts of this rule in the log of that boot.

```
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep -c "non-trusted program"
```

`-b` limits the log to the current boot, and `grep -c` counts the matching lines instead of printing them.

| Situation | Alerts |
|---|---|
| Before the exception (one boot, one login) | 27 |
| After the exception (one boot, one login) | 0 |

**Table 12:** Alerts of the rule before and after tuning

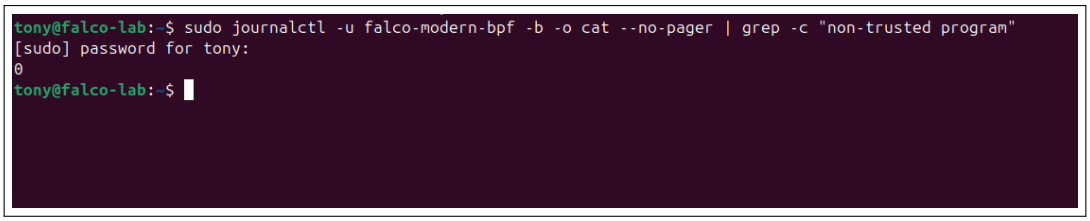



**Figure 9:** Alert count after the exception

A count of zero can mean two things: the exception works, or Falco is not alerting at all. To tell them apart, a deliberate trigger was run after the tuning (see Scenario 1 in the test scenarios). The alert still fired for another program, so the zero is a real result.

**Limits of this measurement.**

- It is a small sample: one boot and one login for each count.
- The two counts were not taken at exactly the same time after the boot, so the procedure is the same but the observation windows are not identical.
- The exception matches the program path only. A replaced file under the same path would be hidden. The package check above protects against this today, and a tighter condition (path and parent process) is a possible improvement.
- The effect on the second rule that uses the same macro was not measured.

# 8 Custom rules

The three default rules examined so far cover reading credentials, taking control with a shell, and running a program from a memory directory. They do not cover everything. A search of the names of the default rules for privilege-related words (`sudo`, `setuid`, `setgid`, `privilege`, `escalat`) found only one rule, which concerns a container (*Debugfs Launched in Privileged Container*). The default rules therefore contain no rule about a user gaining privileges on the host itself. This section documents custom rules written to fill such gaps.

Custom rules are written in the local rules file, below the exception of the first rule, so that the default file stays unchanged. Falco reloads its rules by itself when a rules file is saved (see the configuration section), so every change takes effect at once and has to be validated before it is saved. Each custom rule is documented in the same order: the idea, the rule, its validation, its tests, the noise it produced, the tuning, and its limits.

## 8.1 Custom rule A: setuid or setgid bit set on a file

**The idea**

A program normally runs with the rights of the user who starts it. A file with the *setuid bit* runs with the rights of its *owner*, whoever starts it. System programs such as `sudo` and `passwd` are built this way on purpose, because they need the power of root. The group version of the bit is called *setgid*.

This is also a way to plant a backdoor. An attacker who has root only for a short time can save a spare key for later, as in Table 13. In normal life nobody sets the bit by hand on a new file,

so seeing it set is a strong warning sign. The moment that matters is step 3, long before the attacker uses the file in step 5.

| Step | What happens | Rights of the attacker |
|---|---|---|
| 1 | The attacker gets in as root for a short time. | Root (temporary) |
| 2 | A shell is copied to a hidden place; the copy is owned by root. | Root |
| 3 | The setuid bit is set on the copy (`chmod u+s`). | Root |
| 4 | The access is lost: the hole is fixed or the password changes. | Ordinary user |
| 5 | Later, the attacker runs the copy and gets a root shell. | Root again |

**Table 13:** A setuid backdoor in five steps; Falco can see step 3

### How Falco sees it

Setting the bit is a `chmod` system call whose mode argument contains a flag: `S_ISUID` for setuid and `S_ISGID` for setgid. The rule tests for those flags in the mode that the kernel receives, so it does not matter whether the user writes `chmod u+s` or `chmod 4755`. The event type differs from the earlier rules: it is a *permission change*, whereas the first rule watched a file being opened and the other two watched a program being started. Older Falco rules use a macro called `chmod`, but it does not exist in the default rules file of this installation, so the rule names the system calls itself.

### The rule, first version

```
- rule: Setuid or setgid bit set on a file
  desc: >
    A file was given the setuid or setgid bit, so it now runs with the rights
    of its owner or group. Attackers use this to plant privileged backdoors.
  condition: >
    evt.type in (chmod, fchmod, fchmodat) and evt.dir=<
    and (evt.arg.mode contains "S_ISUID" or evt.arg.mode contains "S_ISGID")
  output: >
    Setuid or setgid bit set on a file | filename=%evt.arg.filename fd=%fd.name
    mode=%evt.arg.mode user=%user.name user_uid=%user.uid
    user_loginuid=%user.loginuid process=%proc.name proc_exepath=%proc.exepath
    parent=%proc.pname command=%proc.cmdline terminal=%proc.tty
  priority: WARNING
  tags: [host, filesystem, mitre_privilege_escalation, T1548.001]
```

- `evt.type in (chmod, fchmod, fchmodat)` selects the three system calls that change permissions: by path, by open file, and by path relative to a directory.
- `evt.arg.mode contains "S_ISUID"` (or `"S_ISGID"`) is the heart of the rule: the new mode contains the setuid or the setgid flag.
- The output starts with a fixed sentence, so that the alerts of this rule can be found with `grep`. The fields give the file, the new mode, the user (with `user_loginuid` for the original account), the program and its path, the parent, the command line, and the terminal.

- The tag `T1548.001` is the MITRE sub-technique *Setuid and Setgid*, carried by Falco's own incubating rule for the same behaviour, on which this condition is modelled.

### Validation and loading

```
sudo falco -V /etc/falco/falco_rules.yaml -V /etc/falco/falco_rules.local.yaml
```

The first validation reported `Ok, with warnings`. The warning, `LOAD_DEPRECATED_ITEM`, says that the field `evt.dir` is deprecated: this version of Falco no longer records the start of a system call, only its end, so the condition `evt.dir=<` is always true and can be removed. The line had been copied from older examples, so examples written for older versions can be out of date. It was removed with:

```
sudo sed -i 's/ and evt.dir=<//' /etc/falco/falco_rules.local.yaml
```

The second validation reported plain `Ok` for both files. The rule was then loaded by restarting the service, and the log showed both rules files loaded with schema validation ok.

**Automatic reload.** The log of the service also showed that Falco had reloaded its rules without a restart each time the file was saved: the same process loaded rules at 11:57 (with the warning) and at 12:02 (without it), and a new process only appeared after the manual restart at 12:04. The setting behind this is `watch_config_files: true` in `falco.yaml`, which makes Falco reload itself when its configuration or rules files change. A rule change is therefore live the moment the file is saved, which is convenient in a lab, and which makes validation before saving important, because a mistake is live at once.

```
tony@falco-lab:~$ sudo journalctl -u falco-modern-bpf -b --no-pager --since "11:
50" --until "12:10" | grep -E "Warnings|Loading rules|Started|Stopped"
[sudo] password for tony:
Oct 03 11:51:51 falco-lab systemd[1]: Started falco-modern-bpf.service - Falco:
Container Native Runtime Security with modern ebpf.
Oct 03 11:51:52 falco-lab falco[871]: Loading rules from:
Oct 03 11:57:25 falco-lab falco[871]: Loading rules from:
                                      1 Warnings:
Oct 03 11:57:26 falco-lab falco[871]: Loading rules from:
                                      1 Warnings:
Oct 03 12:02:03 falco-lab falco[871]: Loading rules from:
Oct 03 12:02:03 falco-lab falco[871]: Loading rules from:
Oct 03 12:04:12 falco-lab systemd[1]: Stopped falco-modern-bpf.service - Falco:
Container Native Runtime Security with modern ebpf.
Oct 03 12:04:12 falco-lab systemd[1]: Started falco-modern-bpf.service - Falco:
Container Native Runtime Security with modern ebpf.
Oct 03 12:04:12 falco-lab falco[3317]: Loading rules from:
tony@falco-lab:~$
```

**Figure 10:** Falco loading its rules when the file is saved, without a restart

### Tests

Four tests were run as an ordinary user on empty files in `/tmp`. The files are harmless: they are empty, they belong to the user, and they were deleted afterwards.

| # | Command | Expected | Observed |
|---|---|---|---|
| 1 | `chmod u+s` on a file | Alert | Alert, `mode` contains `S_ISUID` |
| 2 | `chmod 600` on a file | Silent | Silent: the count of alerts did not change |
| 3 | `chmod 4600` on a file | Alert | Alert, `S_ISUID`: the number form is detected too |
| 4 | `chmod 2600` on a file | Alert | Alert, `S_ISGID`: the group branch works |

**Table 14:** Tests of custom rule A

The first test produced this alert:

```
12:10:19.837471163: Warning Setuid or setgid bit set on a file | filename=/tmp/suid_demo fd=
    mode=S_IROTH|S_IWGRP|S_IRGRP|S_IWUSR|S_IRUSR|S_ISUID user=tony user_uid=1000 user_loginuid
    =1000 process=chmod proc_exepath=/usr/bin/chmod parent=bash command=chmod u+s /tmp/suid_demo
     terminal=34816 container_id=host container_name=host container_image_repository=
    container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

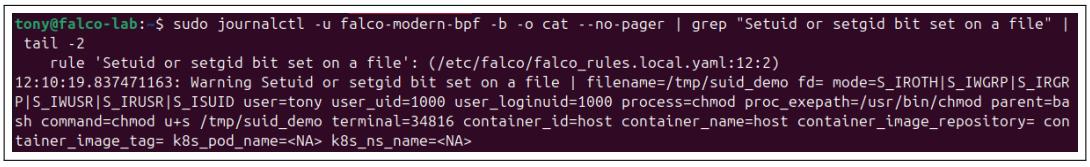



**Figure 11:** Alert raised by `chmod u+s` on a file

- `mode` lists the complete new permissions. The flag `S_ISUID` is the setuid bit; the other flags are the ordinary read and write permissions of the file.
- `process` and `command` show that `chmod` was typed by hand: the parent is a shell and a terminal is attached.
- `fd` is empty, because a `chmod` by path has no open file. The field adds nothing to this rule.

The test with `chmod 4600` produced `mode=S_IWUSR|S_IRUSR|S_ISUID` and the test with `chmod 2600` produced `mode=S_IWUSR|S_IRUSR|S_ISGID`. In both cases the command line shows no letter `s`, and the rule still fired, because it reads the mode and not the text that was typed.




**Figure 12:** Test of the setgid branch: the alert contains `S_ISGID`

### Noise check

Because the rule fires for everyone, including root, legitimate programs that set the bit were expected to trigger it. Before triggering any, the files that already have the setuid bit were listed. The command only reads and does not fire the rule:

```
find / -xdev -perm -4000 -type f 2>/dev/null | sort
```

- `-xdev` stays on the main disk, `-perm -4000` keeps files with the setuid bit, `-type f` keeps regular files, and `2>/dev/null` hides permission errors.

The result was 16 files, all in system folders (Table 15). None was in `/tmp`, a home directory, or `/dev/shm`.

| Group | Programs | Why they need the bit |
|---|---|---|
| Account tools | `passwd`, `chsh`, `chfn`, `gpasswd`, `newgrp` | Change the account of the user, which writes to files owned by root |
| Privilege tools | `su`, `sudo`, `pkexec`, `polkit-agent-helper-1` | Run something as another user |
| Mounting | `mount`, `umount`, `fusermount3` | Attach and detach drives |
| System helpers | `dbus-daemon-launch-helper`, `Xorg.wrap`, `ssh-keysign`, `pppd` | Parts of the desktop, the display, SSH, and networking |

**Table 15:** The 16 files that already had the setuid bit before the noise test

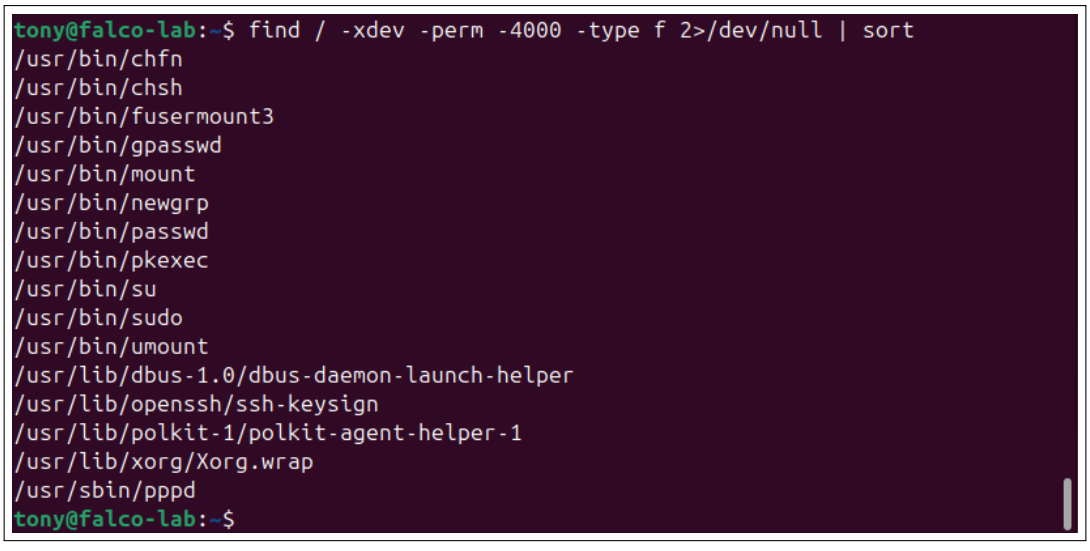



**Figure 13:** Files that already had the setuid bit

Such a list is also a baseline: the rule catches the moment when the bit is set, but only while Falco is running, and a list saved now and compared later would reveal a file that was changed while Falco was off.

**The experiment.** A package that owns one setuid file was reinstalled, as an update would do, and the alerts of the rule were counted before and after.

```
dpkg -S fusermount3
sudo apt install --reinstall -y fuse3
```

The package of the file is `fuse3`. The first attempt, `dpkg -S` with the full path `/usr/bin/fusermount3`, found nothing, because the package database records the file as `/bin/fusermount3` and `dpkg` compares the text without following the link from `/bin` to `/usr/bin`. Searching by name avoids the problem. The reinstall raised one alert, and a second reinstall, run to read more fields, raised another:

```
13:24:54.274825942: Warning Setuid or setgid bit set on a file | filename=/bin/fusermount3 fd=
    mode=S_IXOTH|S_IROTH|S_IXGRP|S_IRGRP|S_IXUSR|S_IWUSR|S_IRUSR|S_ISUID user=root user_uid=0
    user_loginuid=1000 process=chmod proc_exepath=/usr/bin/chmod parent=fuse3.postinst gparent=
    dpkg ggparent=apt gggparent=sudo command=chmod 4755 /bin/fusermount3 terminal=34818
    container_id=host container_name=host container_image_repository= container_image_tag=
    k8s_pod_name=<NA> k8s_ns_name=<NA>
```

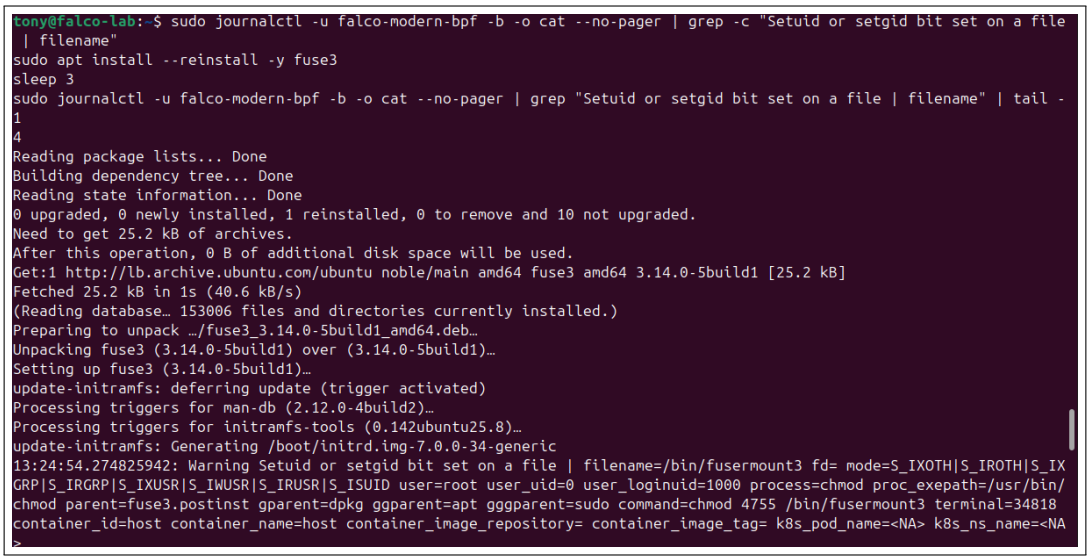



**Figure 14:** Alert raised by the reinstall of a package, with the chain of processes

| Field | Value | Meaning |
|---|---|---|
| `process` | `chmod` | The program that set the bit. |
| `parent` | `fuse3.postinst` | The post-install script of the package. |
| `gparent` | `dpkg` | The package manager is in the chain. |
| `ggparent`, `gggparent` | `apt`, `sudo` | The command typed by the user: `sudo apt install`. |
| `filename` / `command` | `/bin/fusermount3` / `chmod 4755 ...` | The same path that the package database records. |
| `user` / `user_loginuid` | `root` / `1000` | Root through `sudo`, started by the lab user. |

**Table 16:** The chain of processes above the `chmod` of a package install

The expectation before the experiment was that `dpkg` itself would be the process that sets the bit. The alert showed otherwise: the process is `chmod`, run by the post-install script of the package, and `dpkg` is two levels above it. This changes what an exception can test: it has to look at the ancestors and not at the name of the process.

The alert is a **false positive**: the file belongs to the package, the command comes from the install script of that package in an installation started by the user, the file is in a system folder, and the mode is the usual one for such programs.

### Tuning

| Exception | What it says | Weakness |
|---|---|---|
| The 16 file paths | These files may get the bit | A replaced file with a backdoor would stay silent |
| A package manager is an ancestor | Anything installed through the package manager may set bits | A malicious package could set bits too |
| A package manager is an ancestor, and the file is in a system folder | The narrowest of the three; bits set in `/tmp`, `/var/tmp`, `/dev/shm`, or a home directory still alert | See the limits below. *Chosen.* |

**Table 17:** Ways to remove the noise

The default rules file already contains a macro for the first half of the chosen exception: `run_by_package_mgmt_binaries` is true when an ancestor is in the list of package-management programs, which includes `dpkg` and `apt`. The shell rule uses it. The final condition is:

```
condition: >
  evt.type in (chmod, fchmod, fchmodat)
  and (evt.arg.mode contains "S_ISUID" or evt.arg.mode contains "S_ISGID")
  and not (run_by_package_mgmt_binaries
    and (evt.arg.filename startswith "/bin/" or evt.arg.filename startswith "/sbin/"
    or evt.arg.filename startswith "/usr/bin/" or evt.arg.filename startswith "/usr/sbin/"
    or evt.arg.filename startswith "/usr/lib/" or evt.arg.filename startswith "/usr/libexec/")
  )
```

The exception is like a door with two guards, and both must agree before the rule stays silent (Table 18). The output of the rule was also extended with three ancestor fields (`gparent`, `ggparent`, `gggparent`), as in the first rule, so that the chain is visible.

| Guard | Question | Test in the rule |
|---|---|---|
| 1 | Is a package manager above this program? | `run_by_package_mgmt_binaries` |
| 2 | Does the path of the file begin with a system folder? | Six `startswith` tests |

**Table 18:** The two conditions of the exception

**Result.** The same reinstall was run again. The count of alerts of the rule was 5 before and 5 after, so the reinstall no longer fires the rule (Table 19). Two controls then showed that the rule can still detect. A plain `chmod u+s` on a file in `/tmp` raised an alert (count 6). A stand-in program named `dpkg` (a copy of `bash`, as in the shell scenario) was then used to run the same `chmod`: it raised an alert (count 7), with `parent=dpkg`, because the file was in `/tmp` and not in a system folder, so the second guard failed.

| Situation | Count of alerts | Result |
|---|---|---|
| First reinstall, before the exception | 3 to 4 | Alert (noise) |
| Second reinstall, before the exception | 4 to 5 | Alert (noise) |
| Third reinstall, after the exception | 5 to 5 | Silent |
| Control 1: `chmod u+s` in `/tmp` | 5 to 6 | Alert |
| Control 2: stand-in `dpkg`, file in `/tmp` | 6 to 7 | Alert |
| Test of the limit: stand-in `dpkg`, tricky path | 7 to 7 | Silent |

**Table 19:** Counts of alerts of custom rule A during the tuning

```
tony@falco-lab:~$ sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep -c "Setuid or setgid bit set on a file
 | filename"
sudo apt install --reinstall -y fuse3
sleep 3
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep -c "Setuid or setgid bit set on a file | filename"
5
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 1 reinstalled, 0 to remove and 10 not upgraded.
Need to get 25.2 kB of archives.
After this operation, 0 B of additional disk space will be used.
Get:1 http://lb.archive.ubuntu.com/ubuntu noble/main amd64 fuse3 amd64 3.14.0-5build1 [25.2 kB]
Fetched 25.2 kB in 2s (15.8 kB/s)
(Reading database… 153006 files and directories currently installed.)
Preparing to unpack …/fuse3_3.14.0-5build1_amd64.deb…
Unpacking fuse3 (3.14.0-5build1) over (3.14.0-5build1)…
Setting up fuse3 (3.14.0-5build1)…
update-initramfs: deferring update (trigger activated)
Processing triggers for man-db (2.12.0-4build2)…
Processing triggers for initramfs-tools (0.142ubuntu25.8)…
update-initramfs: Generating /boot/initrd.img-7.0.0-34-generic
5
tony@falco-lab:~$
```

**Figure 15:** The same reinstall after the exception: the count does not change

```
tony@falco-lab:~$ cp /bin/bash /tmp/dpkg
/tmp/dpkg -c 'chmod u+s /tmp/ctl_demo; true'
sleep 3
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep "Setuid or setgid bit set on a file | filename" | tail -
1
13:32:53.186994334: Warning Setuid or setgid bit set on a file | filename=/tmp/ctl_demo fd= mode=S_IROTH|S_IWGRP|S_IRGRP
|S_IWUSR|S_IRUSR|S_ISUID user=tony user_uid=1000 user_loginuid=1000 process=chmod proc_exepath=/usr/bin/chmod parent=dpk
g gparent=bash ggparent=gnome-terminal- gggparent=systemd command=chmod u+s /tmp/ctl_demo terminal=34816 container_id=ho
st container_name=host container_image_repository= container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
tony@falco-lab:~$
```

**Figure 16:** Control with a stand-in named `dpkg`: the rule still alerts

### Limits

- **A path can start in a system folder and end elsewhere.** The folder test compares the text at the start of the path. The path `/usr/bin/../../tmp/ctl_demo` begins with `/usr/bin/` but points to `/tmp/ctl_demo`. With the stand-in `dpkg` above the `chmod`, the rule stayed silent (count 7 to 7) although the bit was really set on a file in `/tmp`. The cause was not proven separately, but it is the only difference from the control that alerted. A possible fix is to refuse paths that contain `..`, or to use a field that holds the resolved path.
- **The package manager is recognised by its name.** The stand-in named `dpkg` passed the first guard. A process that pretends to be a package manager can therefore set bits in system folders without an alert.
- **A malicious package.** A package installed through the real package manager is trusted by the exception when its files are in system folders.
- **Only the three calls named in the rule are watched.** Newer variants of the system call, such as `fchmodat2`, were not tested.
- **Falco watches only while it runs.** A bit set while Falco was stopped would only be found by comparing with a saved list.

## 8.2 Custom rule B: a network scanner is started on the host

### The idea

Reconnaissance is the information-gathering stage of an attack. An attacker who is already inside a machine does not yet know it, so they run tools that list the open ports and the services behind them. A tool such as `nmap` does this, and its option `-sV` adds the version of each service. The attacker then looks up those versions for known weaknesses and chooses the next target. The default rules file has no rule for this on the host: it only contains a netcat rule that applies inside containers.

Falco sees only what happens on the machine where it runs, so the rule covers one case: a scanner that is *started on the monitored host*, by an attacker who is already inside. Two other kinds of reconnaissance are invisible to it: a scan launched from the attacker's own computer over the network (the work of a network IDS such as Snort or Suricata), and passive reconnaissance such as reading social networks.

The policy behind the rule is that nobody on a normal machine needs a port scanner, so every start is worth reporting, and the level depends on who started it.

### The policy: the person behind the action

Falco has two fields for the user, and the policy depends on the difference between them (Table 20). `user.uid` is the user whose rights the process has now. `user.loginuid` is the audit user id, the account that originally logged in, which stays the same through `sudo`. Falco's field reference states that when this value is not set, it is shown as `-1`.

| Who starts the tool | user | user.uid | user.loginuid |
|---|---|---|---|
| The lab user, without sudo | tony | 1000 | 1000 |
| The lab user, with sudo | root | 0 | 1000 |
| A real root login | root | 0 | 0 |
| A service, with no login behind it | any | any (0 if root) | -1 |

**Table 20:** The two user fields in four situations

With sudo, the alert shows user=root (user.uid=0), but user.loginuid is still 1000, so it is not a real root login. user.uid would treat the employee as root and lower the alert, while user.loginuid still identifies the person, so the policy uses it. A service has no login, so its value is -1, which is not 0 and therefore falls on the urgent side without extra work. A rule has only one priority, so a policy with two levels needs two rules (Table 21) that share one list of tool names.

| Rule | Condition | Priority | Who it catches |
|---|---|---|---|
| Network scanner started by a non-root login | user.loginuid != 0 | CRITICAL | Ordinary accounts, with or without sudo, and services (-1) |
| Network scanner started by a root login | user.loginuid = 0 | WARNING | A real root login |

**Table 21:** The two tiers of rule B

### The rules

```
- list: network_scanner_binaries
  items: [nmap, masscan, zmap, hping3, nping, unicornscan, nc, ncat, netcat, nc.openbsd, nc.
    traditional]

- rule: Network scanner started by a non-root login
  desc: >
    A network scanning tool was started by an account that did not log in as
    root, or by a process with no login behind it, such as a service. Nobody on
    a normal machine needs a port scanner, so this is treated as urgent.
  condition: >
    spawned_process and proc.name in (network_scanner_binaries)
    and user.loginuid != 0
  output: >
    Network scanner started by a non-root login | user=%user.name
    user_uid=%user.uid user_loginuid=%user.loginuid process=%proc.name
    proc_exepath=%proc.exepath parent=%proc.pname gparent=%proc.aname[2]
    ggparent=%proc.aname[3] gggparent=%proc.aname[4] command=%proc.cmdline
    terminal=%proc.tty exe_flags=%evt.arg.flags
  priority: CRITICAL
  tags: [host, network, mitre_discovery, T1046]

- rule: Network scanner started by a root login
  desc: >
    A network scanning tool was started by an account that logged in as root.
    It is reported at a lower level, but it is still reported.
  condition: >
```

```
    spawned_process and proc.name in (network_scanner_binaries)
    and user.loginuid = 0
  output: >
    Network scanner started by a root login | user=%user.name
    user_uid=%user.uid user_loginuid=%user.loginuid process=%proc.name
    proc_exepath=%proc.exepath parent=%proc.pname gparent=%proc.aname[2]
    ggparent=%proc.aname[3] gggparent=%proc.aname[4] command=%proc.cmdline
    terminal=%proc.tty exe_flags=%evt.arg.flags
  priority: WARNING
  tags: [host, network, mitre_discovery, T1046]
```

- The list `network_scanner_binaries` is defined once and used by both rules, so a tool added to it applies to both. It holds dedicated scanners and the netcat family. The local port lists `ss` and `netstat` were left out of this version on purpose, to measure their noise first.
- `spawned_process` is a macro of the default rules file: a program was started. `proc.name in (...)` tests the name against the list.
- The two conditions differ only in the last line, which tests the login id against 0.
- The output starts with a fixed sentence, so that the alerts can be found with `grep`, and shows both user fields, the full chain of processes, and the full command line.
- The tag `T1046` is the MITRE technique *Network Service Discovery*.

### Validation and loading

The validation reported plain `Ok` for both files, with no warning. Because `Ok` would also appear if the paste had not been saved, the definitions in the file were listed with `grep -n -E "^-(rule|list|macro):"`: five definitions were found (the first exception, rule A, the list, and the two scanner rules). The log of the service then showed that Falco had reloaded its rules by itself, with no warning and no error.

### Tests

No scanner was installed and nothing was scanned. The tests use a harmless stand-in: a copy of the program `true`, which does nothing, saved as `/tmp/nmap`. The rule checks only the name, as in the shell scenario, so the stand-in is enough. Scanning other machines would break the rules of the project, and the stand-in keeps every test on the author's own virtual machine.

```
cp /bin/true /tmp/nmap
/tmp/nmap
sudo /tmp/nmap
sudo systemd-run --wait --collect /tmp/nmap
cp /bin/true /tmp/nmapx
/tmp/nmapx
sudo bash -c 'echo 0 > /proc/self/loginuid; cat /proc/self/loginuid; /tmp/nmap'
/tmp/nmap -sV -p 22,80,443 127.0.0.1
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep "Network scanner started" | tail
     -1
```

- `sudo` gives the process the rights of root while the person behind it stays the same.
- `systemd-run` asks the system manager to run the program as a temporary service. A service is not started from a login session, so it has no login id (`-1`).
- `nmapx` is a name that only looks like a scanner. It is not in the list, and serves as a control.

- Writing `0` to `/proc/self/loginuid` sets the login id of one short-lived root shell to 0, which simulates a real root login. The change applies only to that shell and its children.
- The last test passes the options and the target that a real scan would use. The stand-in ignores them.

| # | Test | Expected | Observed |
|---|---|---|---|
| 1 | Stand-in started by the lab user | `CRITICAL` | `Critical`, `user_loginuid=1000` |
| 2 | The same with `sudo` | `CRITICAL` | `Critical`: `user=root` but `user_loginuid=1000` |
| 3 | Started as a service | `CRITICAL` | `Critical`, `user_loginuid=-1`, `terminal=0` |
| 4 | A normal command, and the lookalike `nmapx` | Silent | Silent: the count of alerts stayed at 3 |
| 5 | Simulated root login | `WARNING` | `Warning`, `user_loginuid=0` |
| 6 | Started with options and a target | `CRITICAL` | `Critical`, with the full command line recorded |

**Table 22:** Tests of custom rule B

The first test produced this alert:

```
19:56:07.052484482: Critical Network scanner started by a non-root login | user=tony user_uid
    =1000 user_loginuid=1000 process=nmap proc_exepath=/tmp/nmap parent=bash gparent=gnome-
    terminal- ggparent=systemd gggparent=systemd command=nmap terminal=34816 exe_flags=
    EXE_WRITABLE container_id=host container_name=host container_image_repository=
    container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```




**Figure 17:** Test 1: the stand-in started by the lab user

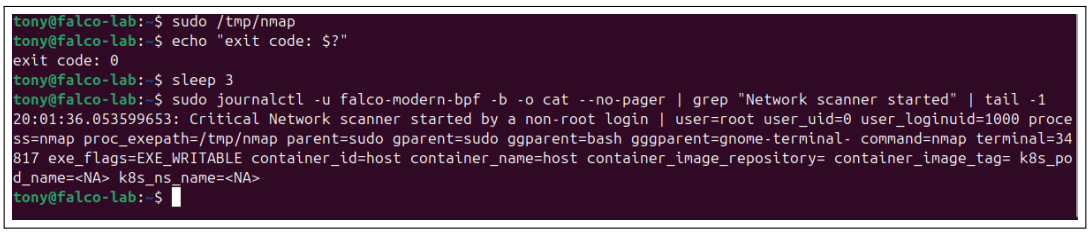



**Figure 18:** Test 2: the same with `sudo`

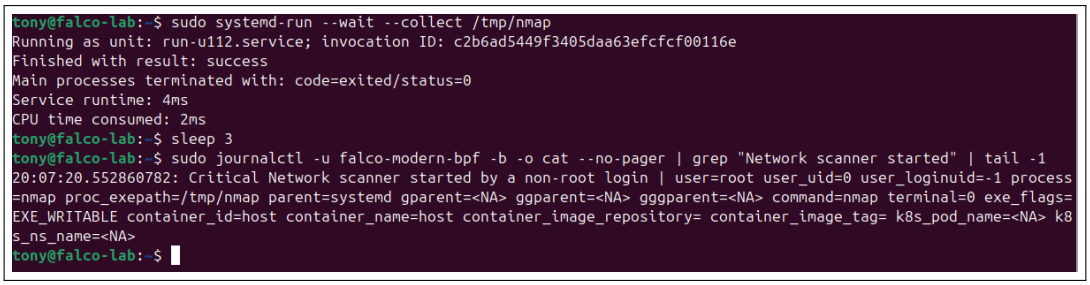



**Figure 19:** Test 3: started as a service, with no login behind it

| # | Who starts it | user / uid | login id | parent | terminal | Priority |
|---|---|---|---|---|---|---|
| 1 | Lab user | tony / 1000 | 1000 | `bash` | 34816 | Critical |
| 2 | Lab user, `sudo` | root / 0 | 1000 | `sudo` | 34817 | Critical |
| 3 | A service | root / 0 | -1 | `systemd` | 0 | Critical |
| 5 | Simulated root login | root / 0 | 0 | `sudo` | 34817 | Warning |

**Table 23:** The alerts of tests 1, 2, 3 and 5 side by side

- **Test 2 is the case that decided the field.** The process runs as root (`user=root`), but `user_loginuid` is still 1000, so the alert stays urgent. Tiering by `user.uid` would have treated it as root and lowered it.
- **Test 3 has the shape of a real attack.** No login, no terminal (`terminal=0`) and the system manager as parent. A scanner started by a hijacked service would look like this. In the shell scenario, the limits noted that a real attack has no terminal, and this is the first test that reproduces it.
- **Test 4 is a control.** A normal command and a name that only resembles a scanner started, Falco saw both, and the rule stayed silent, because the list matches exact names.

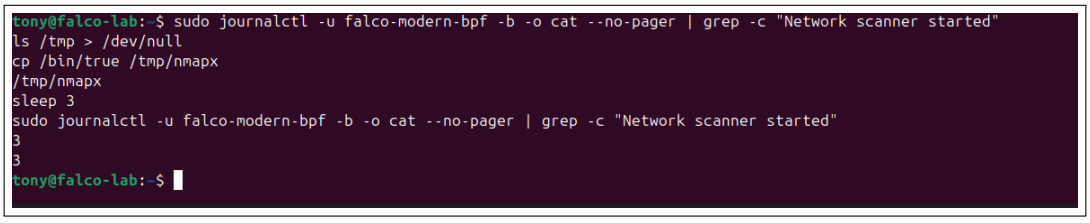



**Figure 20:** Test 4: a normal command and a lookalike name raise no alert

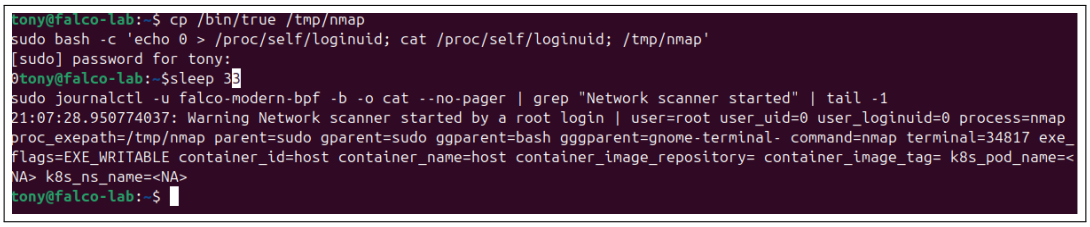



**Figure 21:** Test 5: a simulated root login gives the lower tier

The last test shows what an analyst can read from the alert:

```
21:12:53.128687369: Critical Network scanner started by a non-root login | user=tony user_uid
    =1000 user_loginuid=1000 process=nmap proc_exepath=/tmp/nmap parent=bash gparent=gnome-
    terminal- ggparent=systemd gggparent=systemd command=nmap -sV -p 22,80,443 127.0.0.1
    terminal=34816 exe_flags=EXE_WRITABLE container_id=host container_name=host
    container_image_repository= container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

The field `command` holds the tool, the option `-sV` (service versions), the ports, and the target, so an analyst sees what was probed and what was being looked for. The alert records what was typed and not what happened, so it would look the same for a real scan.

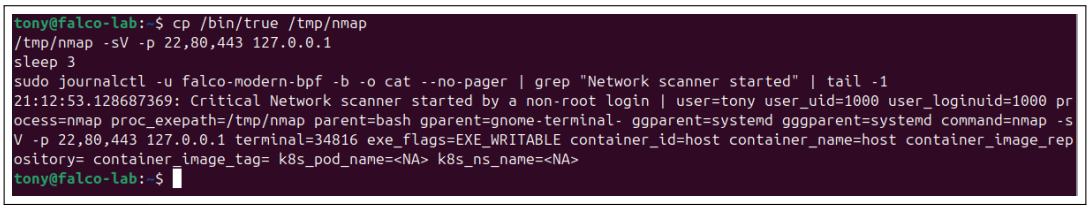



**Figure 22:** Test 6: the alert records the full command line

### Noise check

The rules were loaded at 19:50. About an hour and twenty minutes of normal desktop use later, the alerts of the two scanner rules were listed:

```
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep "Network scanner started" | cut
    -c1-90
```

`cut -c1-90` keeps the first 90 characters of each line, which is enough to show the time, the priority and the start of the sentence. The result was four alerts, at 19:56, 20:01, 20:07 and 21:07, which are the times of tests 1, 2, 3 and 5. Nothing else on the system started a program with one of the names in the list, so the rule produced no noise and needed no tuning in this version. This agrees with the policy: on a normal desktop nobody needs a port scanner.

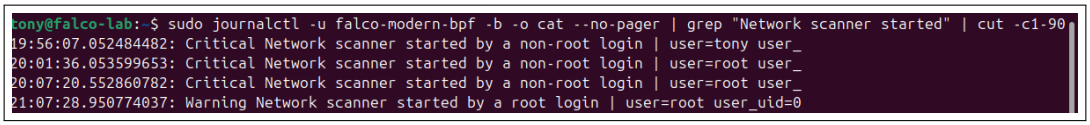



**Figure 23:** Alerts of the scanner rules after about an hour and twenty minutes: only the tests

### Limits

- **A renamed tool escapes the rule.** The list matches exact names. The control `nmapx` stayed silent for the same reason that a real scanner copied under another name would. This is the same weakness as in the shell scenario, seen from the other side.
- **The rule detects the tool and not the scan.** A real port scan is many connections in a short time, and a Falco rule looks at one event at a time, so it cannot count them.
- **Only scans started on the monitored host are seen.** A scan from another machine, and passive reconnaissance, are outside what Falco observes.
- **The login id can be changed by a privileged process.** In test 5 the login id of one shell was set to 0 by hand. An attacker who already has root could do the same before starting a scanner, and the alert would drop from `CRITICAL` to `WARNING`. The alert would still be raised, and the chain of processes would still show `sudo`.

- **The tests used a stand-in and not a real scanner,** so they test the rule and not the tool. The `ss` and `netstat` commands were not measured, and the sample is one boot of desktop use.

## 8.3 Custom rule C: tampering with Falco

### The idea

Falco protects a machine only while its rules are intact and the service is running. An attacker who gets root therefore has a reason to blind it first, and there are three ways to do it (Table 24). Because Falco reloads its rules by itself when a file is saved, a changed rule is live at once, which raises the stakes. The mechanism that tunes a rule, an override in the local rules file, can also be used to silence it.

| Way to blind Falco | What Falco sees | Rule |
|---|---|---|
| Edit the rules or the configuration | A program opens a file under `/etc/falco/` for writing, or tries and is refused | C2 (success), C1 (denied) |
| Replace or delete the files | A rename, a move, or a deletion under `/etc/falco/`, with no write inside the folder | C3 |
| Stop or disable the service | `systemctl` starts, with `stop`, `disable` or `mask` and the name of the service | C4 |

**Table 24:** Three ways to blind Falco and the rule that watches each

The default rules file contains no mention of `/etc/falco` or of `falcoctl`, so none of this is covered by default. The four rules carry the MITRE tag `T1562.001`, *Impair Defenses: Disable or Modify Tools*. The priorities follow the ladder of Falco's levels, from `DEBUG` to `EMERGENCY`, with a reason for each step (Table 25).

| Rule | Priority | Reason |
|---|---|---|
| C1 | `CRITICAL` | An account that has no right to change these files is trying to, which is a strong sign of probing. |
| C2 | `WARNING` | A successful write needs root, and an administrator makes legitimate changes, so it is a change to audit. |
| C3 | `WARNING` | The same kind of event as C2: a change made with root. |
| C4 | `ALERT` | Stopping Falco switches off every other alert, so it is placed above `CRITICAL`. `EMERGENCY`, which means that the system is unusable, is left unused. |

**Table 25:** Priorities of the rules of custom rule C

### C2: a Falco file is written

The fields that a rule needs were checked first with `falco -list=syscall`: `evt.res` (the result of the call, `SUCCESS` or an error code), `evt.is_open_write`, and `fs.path.name` (the path of the file, fully resolved). The first version of the rule used the default macro `open_write` and tested that the path starts with `/etc/falco/`. The final version, after the tuning below, is:

```
- rule: Falco configuration or rules file written
  desc: >
    A program opened a file under /etc/falco/ for writing. An attacker can blind
    Falco by editing its rules or configuration, for example to disable a rule.
    Legitimate writers (the update helper falcoctl, package updates, an
    administrator) also appear, and are tuned after they have been measured.
  condition: >
    open_write and fd.name startswith "/etc/falco/"
    and not (proc.exepath = "/usr/bin/nano" and fd.name endswith ".swp")
  output: >
    Falco configuration or rules file written | file=%fd.name user=%user.name
    user_uid=%user.uid user_loginuid=%user.loginuid process=%proc.name
    proc_exepath=%proc.exepath parent=%proc.pname gparent=%proc.aname[2]
    ggparent=%proc.aname[3] gggparent=%proc.aname[4] command=%proc.cmdline
    terminal=%proc.tty
  priority: WARNING
  tags: [host, filesystem, mitre_defense_evasion, T1562.001]
```

The validation reported plain `Ok` for both files, and the log showed that the rules were reloaded with no warning. The first test appended a comment line to the rules file through `sudo`. A comment is ignored by Falco, so the file stays valid, and the line was removed afterwards.

```
sudo sh -c 'echo "# rule C test line" >> /etc/falco/falco_rules.local.yaml'
```

The `sudo sh -c '...'` form makes the root shell do the redirection. With `sudo echo ... » file`, the redirection would be made by the shell of the user, without the rights of root, and would be refused.

```
11:51:53.647252196: Warning Falco configuration or rules file written | file=/etc/falco/
    falco_rules.local.yaml user=root user_uid=0 user_loginuid=1000 process=sh proc_exepath=/usr/
    bin/dash parent=sudo gparent=sudo ggparent=bash gggparent=gnome-terminal- command=sh -c echo
     \"# rule C test line\" >> /etc/falco/falco_rules.local.yaml terminal=34817 container_id=
    host container_name=host container_image_repository= container_image_tag= k8s_pod_name=<NA>
    k8s_ns_name=<NA>
```




**Figure 24:** C2: a write to the rules file through `sudo`

The process is the root shell (`sh`, which is `dash`), because the redirection opens the file and not `echo`. The text that was written is visible, because it is on the command line. When the change is made in an editor, the command line shows only the editor and the file, so Falco records that a file was written and not what was written.

**A second way to edit: `sed -i`.** The command `sed -i '/text/d' file` edits a file in place, but it writes a temporary copy in the same folder and then renames it over the original. The alert of C2 showed the temporary name (`file=/etc/falco/sedq2YxJQ`), and the rename that replaced the real file raised nothing. This is a limit of C2 and the reason for C3.

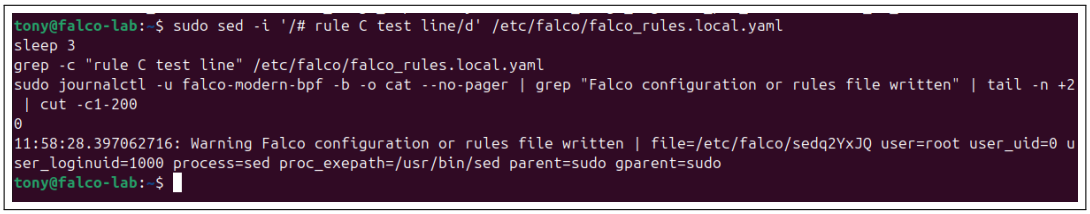



**Figure 25:** C2: `sed -i` shows a temporary file name

### C1: a denied attempt to write

A normal user cannot write to these files, because they belong to root. The kernel refuses, but Falco still sees the request and its answer (`EACCES`). The test was run without `sudo`:

```
echo "# attempt" >> /etc/falco/falco_rules.local.yaml
```

The result was `Permission denied`, and the count of C2 alerts did not change (2 before and 2 after). C2 uses the default macro `open_write`, which only matches opens that succeeded (it requires `fd.num>=0`), so a refused attempt needs its own rule. The default file has a macro for failed opens, `open_file_failed`, which it describes as useful to detect threat actors making mistakes.

```
- rule: Denied write attempt on a Falco configuration or rules file
  desc: >
    A program tried to open a file under /etc/falco/ for writing and the kernel
    refused. These files belong to root, so an ordinary account that tries to
    change them is probing, and is treated as urgent.
  condition: >
    evt.type in (open, openat, openat2) and evt.is_open_write=true
    and evt.res startswith E and fs.path.name startswith "/etc/falco/"
  output: >
    Denied write attempt on a Falco file | file=%fs.path.name evt_res=%evt.res
    user=%user.name user_uid=%user.uid user_loginuid=%user.loginuid
    process=%proc.name proc_exepath=%proc.exepath parent=%proc.pname
    gparent=%proc.aname[2] ggparent=%proc.aname[3] gggparent=%proc.aname[4]
    command=%proc.cmdline terminal=%proc.tty
  priority: CRITICAL
  tags: [host, filesystem, mitre_defense_evasion, T1562.001]
```

The condition tests that the call is an open for writing, that the result is an error code (every error code starts with `E`), and that the resolved path is in Falco's folder. The field `fs.path.name` was not certain to work for a failed open, which has no file descriptor, and the test showed that it does.

```
12:10:17.523733800: Critical Denied write attempt on a Falco file | file=/etc/falco/falco_rules.
    local.yaml evt_res=EACCES user=tony user_uid=1000 user_loginuid=1000 process=bash
    proc_exepath=/usr/bin/bash parent=gnome-terminal- gparent=systemd ggparent=systemd gggparent
    =<NA> command=bash terminal=34816 container_id=host container_name=host
    container_image_repository= container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

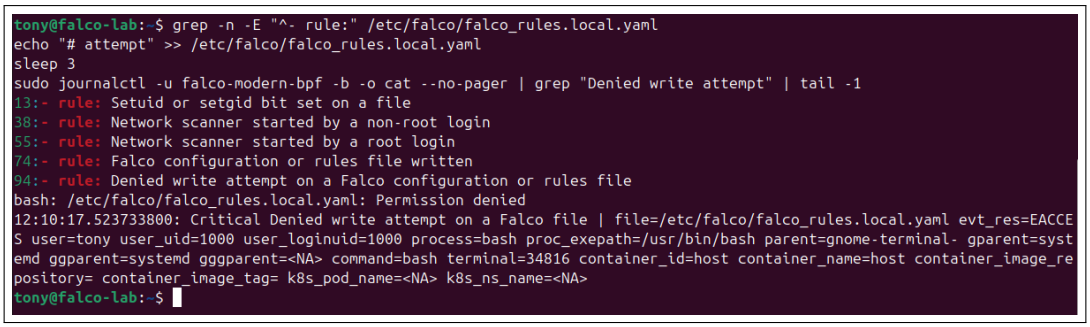



**Figure 26:** C1: a denied write by an ordinary user

The process is `bash`, because the shell does the redirection, and the command is only `bash`, so the alert shows the file and who tried but not the text. A control showed that the two rules do not overlap: a successful write through `sudo` raised C2 and left the count of C1 at 1.

### Noise of C2 and the tuning

All the alerts of C2 in the boot were listed (Table 26). One edit in `nano` raised three of them, and only the last one names the real file: `nano` creates a hidden swap file next to the file it edits, which works as a lock while the file is open, and it wrote that file twice before saving.

| # | Time | File in the alert | Cause |
|---|---|---|---|
| 1 | 11:51:53 | The rules file | `echo` through `sudo` |
| 2 | 11:58:28 | `sedq2YxJQ` | `sed -i`: the temporary copy |
| 3 | 12:07:30 | `.falco_rules.local.yaml.swp` | `nano`: the swap file |
| 4 | 12:07:34 | `.falco_rules.local.yaml.swp` | `nano`: the swap file again |
| 5 | 12:08:14 | The rules file | `nano` saving the real file |
| 6 | 12:14:31 | The rules file | `echo` through `sudo` (a control) |
| 7 | 12:14:34 | `sedN3srQq` | `sed -i`: the temporary copy |

**Table 26:** The alerts of C2 in one boot, and their causes

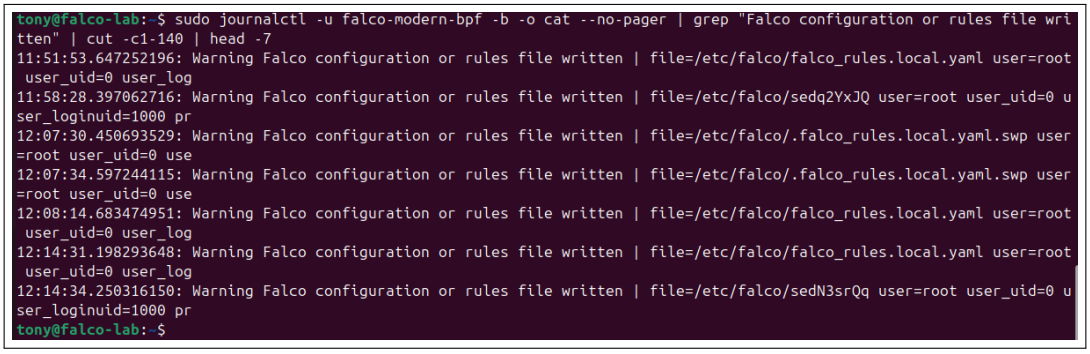



**Figure 27:** The first seven alerts of C2, with the swap files of `nano`

All of them show `user=root` and `user_loginuid=1000`, so C2 cannot tell an edit by the administrator from an edit by an attacker who uses `sudo`. It records that a change happened, with who and with what. The swap-file alerts are pure noise, since the real write is reported by alert

5. The exception in the final rule removes them and keeps everything else, with two conditions that must both be true: the program is `nano` (tested by its path) and the file name ends in `.swp`.

The effect was measured with one `nano` edit: three alerts before the exception, and one after it (the count went from 8 to 9, and the alert named the real file). A control showed that the exception is narrow: an `echo` through `sudo` still raised the alert.

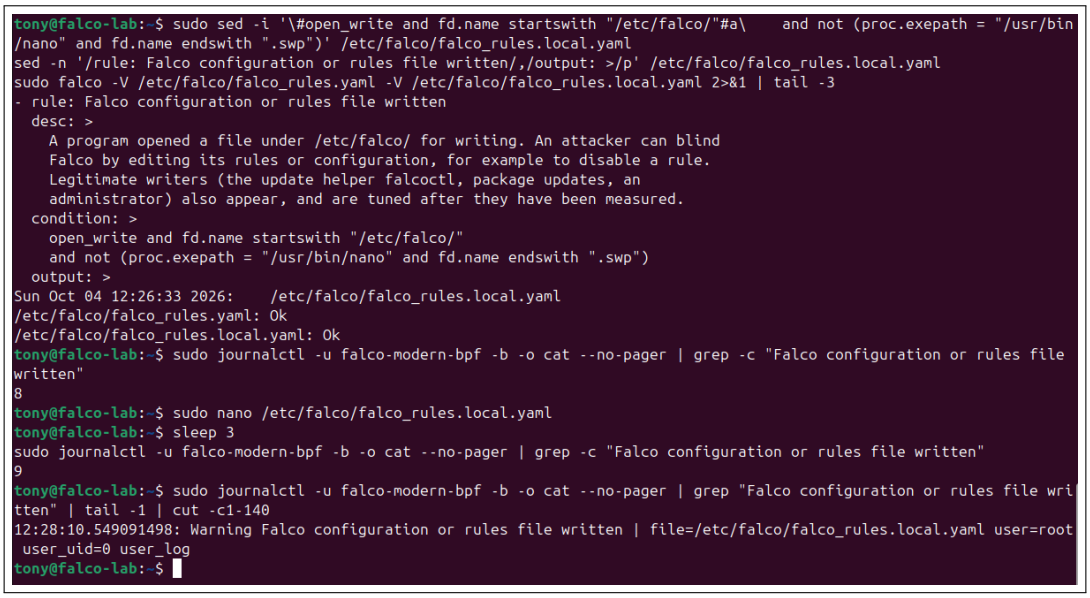



**Figure 28:** One `nano` edit after the exception: one alert

**A counting mistake.** After adding C1, the count of C2 alerts was 6 and not 3 as expected, because the `nano` edit that added the rule had also raised C2. The edits of the author trigger the rule, so a count has to include them. The listing above explains every alert.

### C3: a rename, a move, or a deletion

The prediction was that a file can be put into Falco's folder without any write inside it. The test created a file in `/tmp` and moved it in:

```
echo "# c3 test" > /tmp/c3_test.txt
sudo mv /tmp/c3_test.txt /etc/falco/c3_test.txt
```

The two paths are on the same disk (`df` showed `/dev/sda2` for both), so `mv` is a pure rename. The name `c3_test.txt` is not one of the files that Falco loads, so nothing was reloaded. The count of C2 alerts was 0 before and 0 after: C2 does not see a rename into its folder. Falco has fields for the two ends of a rename (`fs.path.source` and `fs.path.target`), and for the path of a deletion (`fs.path.name`).

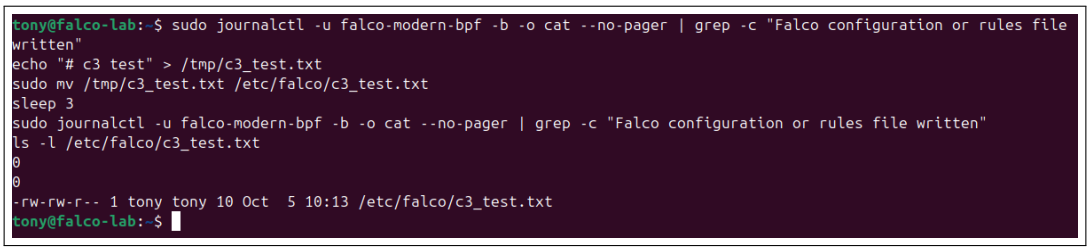



**Figure 29:** The gap: a file moved into `/etc/falco/` raises no alert from C2

```
- rule: Falco configuration or rules file renamed or deleted
  desc: >
    A file under /etc/falco/ was renamed, moved into place, or deleted. A write
    is not needed for this, so the rule that watches writes does not see it.
  condition: >
    (evt.type in (rename, renameat, renameat2) and
     (fs.path.target startswith "/etc/falco/" or fs.path.source startswith "/etc/falco/"))
    or
    (evt.type in (unlink, unlinkat, rmdir) and fs.path.name startswith "/etc/falco/")
  output: >
    Falco file renamed or deleted | evt=%evt.type source=%fs.path.source
    target=%fs.path.target file=%fs.path.name user=%user.name
    user_uid=%user.uid user_loginuid=%user.loginuid process=%proc.name
    proc_exepath=%proc.exepath parent=%proc.pname gparent=%proc.aname[2]
    ggparent=%proc.aname[3] gggparent=%proc.aname[4] command=%proc.cmdline
    terminal=%proc.tty
  priority: WARNING
  tags: [host, filesystem, mitre_defense_evasion, T1562.001]
```

```
10:24:51.141217378: Warning Falco file renamed or deleted | evt=renameat2 source=/tmp/c3_test2.
    txt target=/etc/falco/c3_test2.txt file=<NA> user=root user_uid=0 user_loginuid=1000 process
    =mv proc_exepath=/usr/bin/mv parent=sudo gparent=sudo ggparent=bash gggparent=gnome-terminal
    - command=mv /tmp/c3_test2.txt /etc/falco/c3_test2.txt terminal=34817 container_id=host
    container_name=host container_image_repository= container_image_tag= k8s_pod_name=<NA>
    k8s_ns_name=<NA>
```

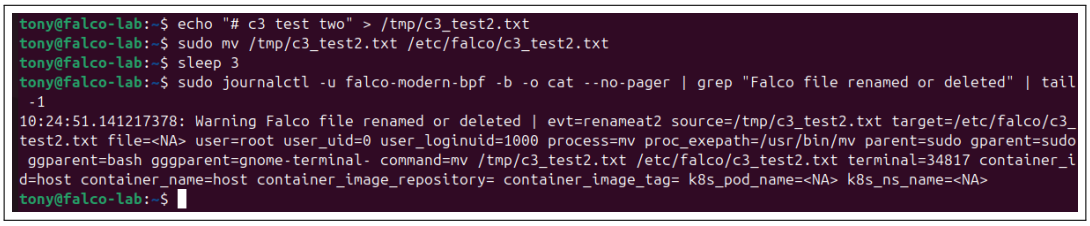



**Figure 30:** C3: a file moved into `/etc/falco/`

In Linux a move and a rename are the same operation: the path of the file changes. `mv` between two folders therefore appears as one rename call (`renameat2`), with the old path in `source` and the new one in `target`, and `file` is empty. For a deletion it is the opposite: `file` holds the path, and `source` and `target` are empty.

| Action | C3 | Fields |
|---|---|---|
| A file moved into `/etc/falco/` | Alert, `renameat2` | `source=/tmp/c3_test2.txt`, `target=/etc/falco/c3_test2.txt`, `process=mv` |
| A file deleted with `rm` | Alert, `unlinkat` | `file=/etc/falco/c3_test2.txt`, `process=rm` |
| An `nano` session with nothing saved | Alert, `unlink` | `file=/etc/falco/.falco_rules.local.yaml.swp`, `process=nano` |

**Table 27:** Tests of C3

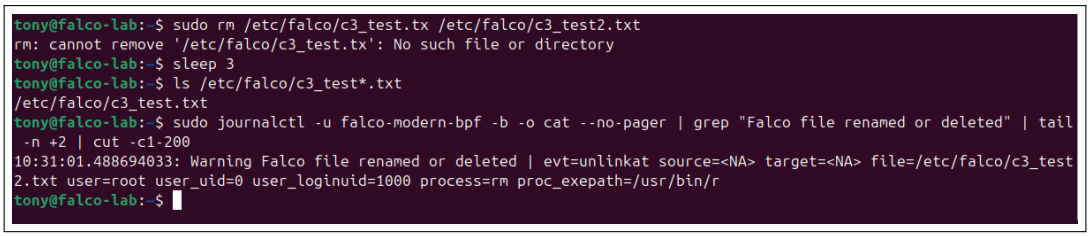



**Figure 31:** C3: a file deleted in `/etc/falco/`

The third test shows the noise of C3: opening the rules file in `nano` and leaving without saving raised one alert, because `nano` deletes its swap file when it exits. This alert was kept on purpose, as an audit trail of editor sessions: someone opened Falco's rules, even if nothing was saved. It does not detect everything, since reading the file with `cat` raises nothing and only an `nano` session is visible. A real change made in `nano` raises C2 for the real write and C3 for the swap file.

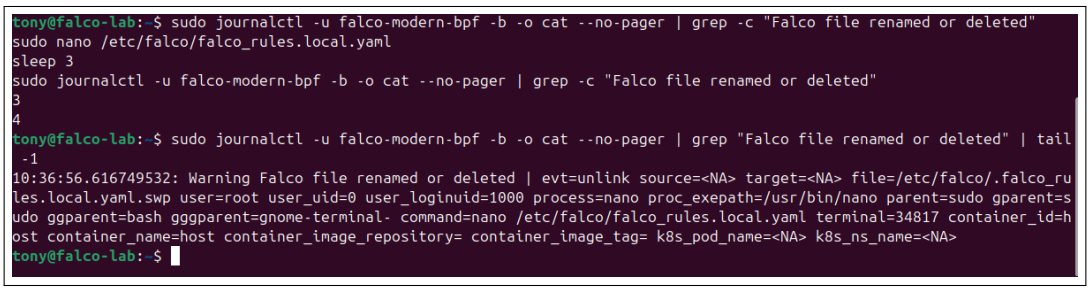



**Figure 32:** C3: an `nano` session with nothing saved

### C4: the service is stopped or disabled

Even with its files intact, Falco can be switched off with `systemctl stop`, `disable` or `mask`. The rule watches for `systemctl` starting with one of those verbs and the name of a Falco service. `restart` is not in the list on purpose, because a restart does not blind Falco for long.

```
- rule: Falco service stopped or disabled
  desc: >
    systemctl was used to stop, disable, mask or kill a Falco service. Nobody
    has a normal reason to do this, even as root, because it switches off the
    monitoring. Restarts are not reported.
  condition: >
    spawned_process and proc.name = systemctl
    and proc.cmdline contains "falco"
    and (proc.cmdline contains " stop" or proc.cmdline contains " disable"
    or proc.cmdline contains " mask" or proc.cmdline contains " kill")
  output: >
    Falco service stopped or disabled | user=%user.name user_uid=%user.uid
    user_loginuid=%user.loginuid process=%proc.name proc_exepath=%proc.exepath
    parent=%proc.pname gparent=%proc.aname[2] ggparent=%proc.aname[3]
    gggparent=%proc.aname[4] command=%proc.cmdline terminal=%proc.tty
  priority: ALERT
  tags: [host, process, mitre_defense_evasion, T1562.001]
```

The priority `ALERT` was accepted by Falco. Three tests were run (Table 28).

| # | Test | Result |
|---|---|---|
| 1 | A stand-in named `systemctl` (a copy of `true`) run with `disable falco-modern-bpf` | `Alert`, with `proc_exepath=/tmp/systemctl`; the real service was not touched |
| 2 | A real `systemctl stop`, then `start` three seconds later | `Alert` written at 10:55:56, with `/usr/bin/systemctl`, `user=root` and `user_loginuid=1000`; Falco was `active` again afterwards |
| 3 | `systemctl status` and `systemctl restart` | Silent: the count of alerts stayed at 2 |

**Table 28:** Tests of C4

```
10:53:51.871626214: Alert Falco service stopped or disabled | user=tony user_uid=1000
    user_loginuid=1000 process=systemctl proc_exepath=/tmp/systemctl parent=bash gparent=gnome-
    terminal- ggparent=systemd gggparent=systemd command=systemctl disable falco-modern-bpf
    terminal=34816 container_id=host container_name=host container_image_repository=
    container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

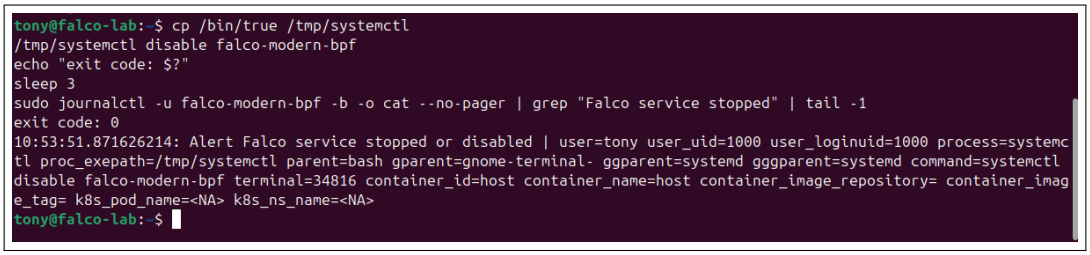



**Figure 33:** C4: the stand-in with the arguments of a `disable`

The second test answers a timing question: when Falco is being stopped, is the alert saved before it stops? In this test it was: the alert is in the log even though the service was shut down right afterwards. It is one sample, so it does not prove that this always happens.

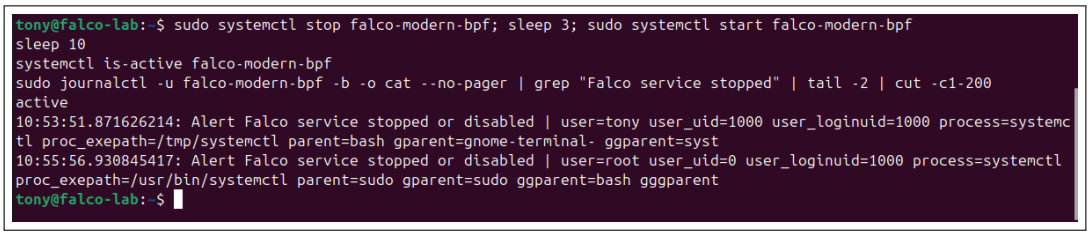



**Figure 34:** C4: a real stop, followed by a start

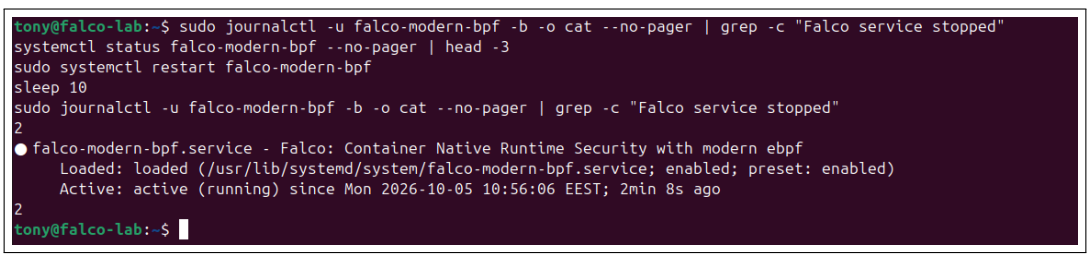



**Figure 35:** C4: `status` and `restart` raise no alert

### Limits

- **C3 and C4 watch only some tools.** C4 watches `systemctl`. A `kill` or `pkill` of the Falco process is not covered, and `service falco stop` was not tested.
- **Reading is not watched.** The rules rely on writes, renames, deletions and the service. Reading the rules file leaves no alert.
- **The alerts do not show what changed.** Falco records that a file was written, and not its content, except when the text is on the command line.
- **An administrator and an attacker with `sudo` look the same** in C2 and C3. The rules work as change detectors and an audit trail.
- **The exception covers only `nano`.** A swap file written by another editor would still alert.
- **The alerts are stored on the same machine.** An attacker with root could try to erase them. Real setups also send alerts off the machine.
- **While Falco is stopped, nothing is watched.** The window in the test was about three seconds.
- **A package update of Falco** stops the service on purpose and was not tested, to avoid prompts about modified configuration files. The update helper `falcoctl` was not reviewed, so its writes, if any, are not measured.

## 8.4 Custom rule D: attack tools

### The idea

Some tools exist only to attack, and nobody on a normal machine needs them. When one of them starts, Falco alerts. The rule has two groups (Table 29). It connects with the first default rule: an attacker who has read the password file with the hashes can turn them back into passwords with a cracking tool.

| Group | Tools | What they do |
|---|---|---|
| Cracking passwords | `john`, `hashcat`, `hydra`, `medusa`, `ncrack` | `john` and `hashcat` turn password hashes back into passwords. `hydra`, `medusa` and `ncrack` try many passwords against a login. |
| Break-in toolkits | `msfconsole`, `msfvenom`, `sqlmap` | Ready-made tools to find and exploit weaknesses in systems. |

**Table 29:** The two groups of attack tools

As for the scanners of custom rule B, Falco sees a tool that *starts on the monitored host*. An attack launched from another machine against this one is not visible to it, and the alert records that the tool started and not what it did, except for what appears on the command line.

### The policy

The policy is the one of rule B, one step higher: the tiers follow the login id (Table 30), and `sudo` does not lower the alert, because the login id still identifies the person. The levels are higher because scanners have legitimate users, such as administrators who scan their own networks, while these tools have almost none on a normal machine. `ALERT` is also the level of C4, and both are cases that need an immediate answer.

| Rule | Condition | Priority | Who it catches |
|---|---|---|---|
| Attack tool started by a non-root login | `user.loginuid != 0` | `ALERT` | Ordinary accounts, with or without `sudo`, and services (`-1`) |
| Attack tool started by a root login | `user.loginuid = 0` | `CRITICAL` | A real root login |

**Table 30:** The two tiers of rule D

### The rules

```
- list: password_cracking_binaries
  items: [john, hashcat, hydra, medusa, ncrack]

- list: exploitation_toolkit_binaries
  items: [msfconsole, msfvenom, sqlmap]

- list: attack_tool_binaries
  items: [password_cracking_binaries, exploitation_toolkit_binaries]

- rule: Attack tool started by a non-root login
  desc: >
    A password-cracking tool or a break-in toolkit was started by an account that
    did not log in as root, or by a process with no login behind it. Nobody on a
    normal machine needs these tools, so this is treated as the most urgent case.
  condition: >
    spawned_process and proc.name in (attack_tool_binaries)
    and user.loginuid != 0
  output: >
    Attack tool started by a non-root login | user=%user.name
    user_uid=%user.uid user_loginuid=%user.loginuid process=%proc.name
    proc_exepath=%proc.exepath parent=%proc.pname gparent=%proc.aname[2]
    ggparent=%proc.aname[3] gggparent=%proc.aname[4] command=%proc.cmdline
    terminal=%proc.tty
  priority: ALERT
  tags: [host, process]

- rule: Attack tool started by a root login
  desc: >
    A password-cracking tool or a break-in toolkit was started by an account that
    logged in as root. It is reported one level lower, but still as critical.
  condition: >
    spawned_process and proc.name in (attack_tool_binaries)
    and user.loginuid = 0
  output: >
    Attack tool started by a root login | user=%user.name
    user_uid=%user.uid user_loginuid=%user.loginuid process=%proc.name
    proc_exepath=%proc.exepath parent=%proc.pname gparent=%proc.aname[2]
    ggparent=%proc.aname[3] gggparent=%proc.aname[4] command=%proc.cmdline
```

```
    terminal=%proc.tty
  priority: CRITICAL
  tags: [host, process]
```

- Two lists name the groups, and a third list contains both, so the rules use a single name. A Falco list can hold other lists, as the default rules file does.
- `spawned_process and proc.name in (...)` means that a program started and that its name is in the list. It is the same test as in the scanner rule.
- The two rules differ only in the login id and in the priority.
- No MITRE tag was added to this rule.

### Validation and loading

The validation reported plain `Ok` for both files. The definitions in the file were listed to check that the paste had been saved: the file contained 14 definitions, the last five being the three lists and the two rules of D. The log showed that Falco had reloaded its rules by itself, with no warning and no error.

### Tests

No attack tool was needed for most tests. A harmless stand-in was used: a copy of the program `true`, which does nothing, saved under the name of the tool. When the stand-in runs, the process takes its name from the file, and the rule tests only the name, so Falco reacts as if the tool had started. This tests the rule and not the tool. Nothing was installed, and nothing was attacked.

```
cp /bin/true /tmp/john
/tmp/john
cp /bin/true /tmp/msfconsole
/tmp/msfconsole
sudo /tmp/john
cp /bin/true /tmp/johnx
/tmp/johnx
sudo bash -c 'echo 0 > /proc/self/loginuid; cat /proc/self/loginuid; /tmp/john'
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep "Attack tool started" | tail -1
```

- `john` and `msfconsole` are names from the first and the second group. `sudo` gives the process the rights of root, while the person behind it stays the same.
- `johnx` is a name that only resembles a tool and is not in the list. It serves as a control, with a normal command.
- Writing `0` to `/proc/self/loginuid` sets the login id of one short-lived root shell to 0, which simulates a real root login. The change applies only to that shell and its children.

| # | Test | Expected | Observed |
|---|---|---|---|
| 1 | Stand-in `john` started by the lab user | `ALERT` | `Alert`, `user_loginuid=1000` |
| 2 | The real tool `hydra`, with its help option | `ALERT` | `Alert`, `proc_exepath=/usr/bin/hydra` |
| 3 | Stand-in `msfconsole` (second group) | `ALERT` | `Alert`, the same rule |
| 4 | The stand-in `john` with `sudo` | `ALERT` | `Alert`: `user=root`, `user_loginuid=1000` |
| 5 | A normal command, and the lookalike `johnx` | Silent | Silent: the count of alerts stayed at 4 |
| 6 | Simulated root login | `CRITICAL` | `Critical`, `user_loginuid=0` |

**Table 31:** Tests of custom rule D

The first test produced this alert:

```
22:36:10.230993720: Alert Attack tool started by a non-root login | user=tony user_uid=1000
    user_loginuid=1000 process=john proc_exepath=/tmp/john parent=bash gparent=gnome-terminal-
    ggparent=systemd gggparent=systemd command=john terminal=34816 container_id=host
    container_name=host container_image_repository= container_image_tag= k8s_pod_name=<NA>
    k8s_ns_name=<NA>
```

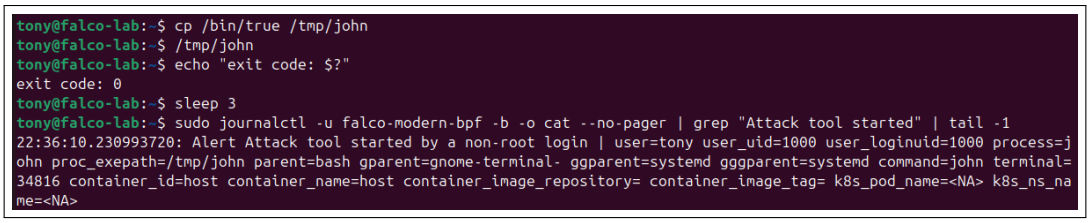



**Figure 36:** Test 1: the stand-in `john` started by the lab user

**A real tool.** One tool was installed to confirm that a stand-in behaves like the real thing: `hydra`, which needed 21 packages and 6,809 kB of download (23.1 MB on disk). It was started only with its help option, on the virtual machine of the author, and it was not pointed at anything. The alert has the same shape as the one of the stand-in, but the program is a real one from a system folder, and the command line shows what was typed. The installation raised no alert from custom rule A, because the tool has no setuid file.

```
22:40:31.958807014: Alert Attack tool started by a non-root login | user=tony user_uid=1000
    user_loginuid=1000 process=hydra proc_exepath=/usr/bin/hydra parent=bash gparent=gnome-
    terminal- ggparent=systemd gggparent=systemd command=hydra -h terminal=34816 container_id=
    host container_name=host container_image_repository= container_image_tag= k8s_pod_name=<NA>
    k8s_ns_name=<NA>
```

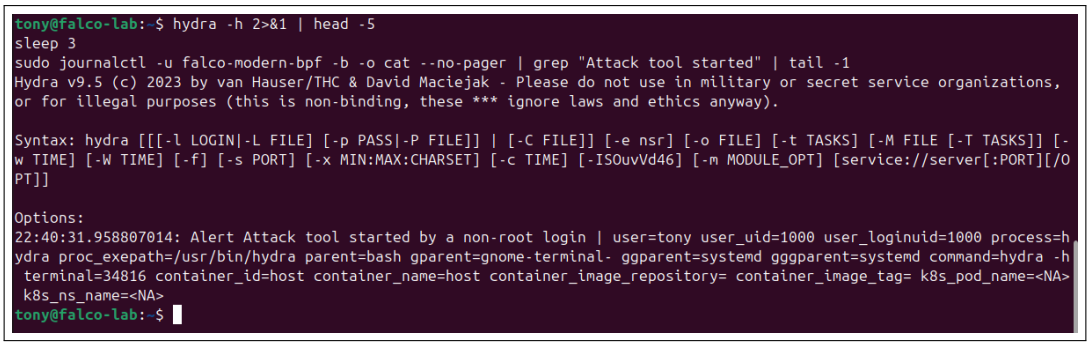



**Figure 37:** Test 2: the real tool `hydra`, started with its help option

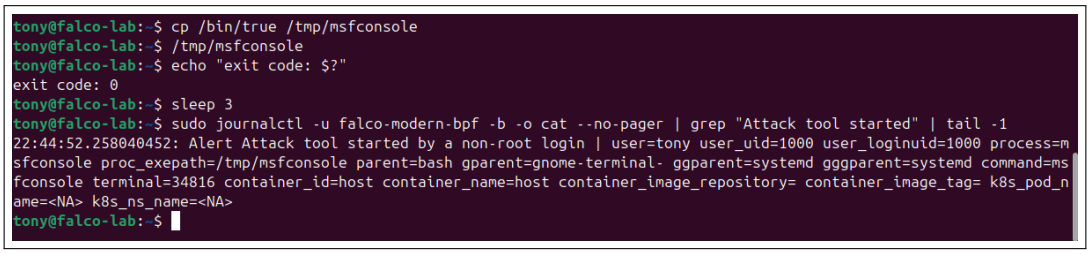



**Figure 38:** Test 3: the stand-in `msfconsole`, from the second group

| # | Who starts the tool | user / uid | login id | Priority |
|---|---|---|---|---|
| 1 | The lab user | tony / 1000 | 1000 | Alert |
| 2 | The lab user, real `hydra` | tony / 1000 | 1000 | Alert |
| 4 | The lab user, with `sudo` | root / 0 | 1000 | Alert |
| 6 | A simulated root login | root / 0 | 0 | Critical |

**Table 32:** The alerts of tests 1, 2, 4 and 6 side by side

Test 4 shows again why the tiers use the login id: the process runs as root, but the person behind it is still the lab user, so the alert stays at the urgent level. With `user.uid`, it would have been lowered.

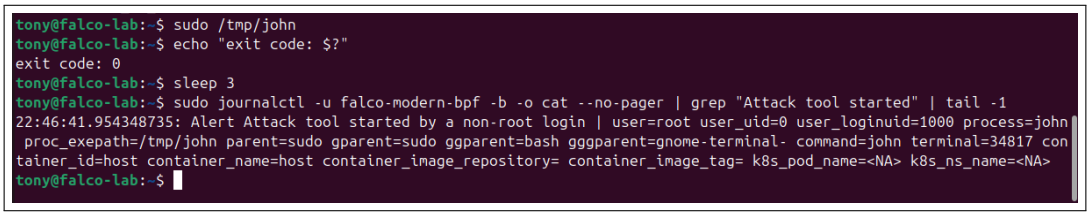



**Figure 39:** Test 4: the stand-in with `sudo`, with the login id still 1000

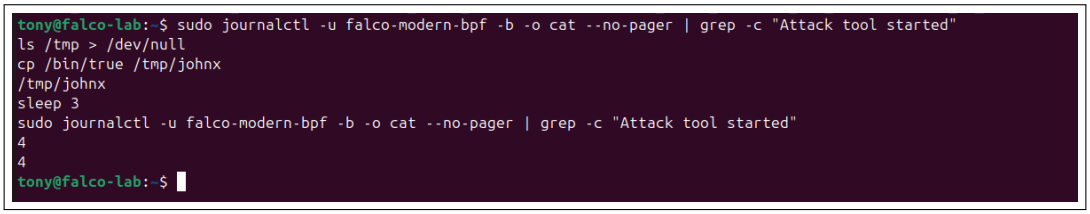



**Figure 40:** Test 5: a normal command and a lookalike name raise no alert

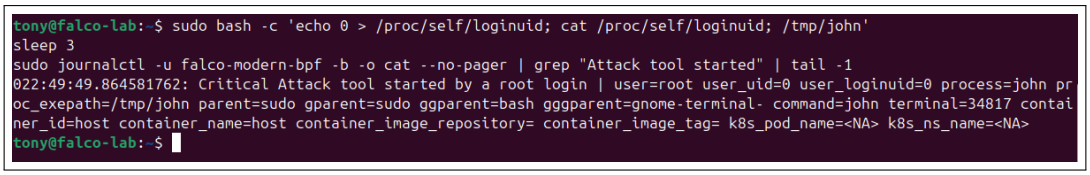



**Figure 41:** Test 6: a simulated root login gives the lower tier

### Noise check

After the tests, the alerts of the two rules were listed with their times:

```
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep "Attack tool started" | cut -c1
    -95
```

The result was five alerts, at 22:36, 22:40, 22:44, 22:46 and 22:49, which are the times of tests 1, 2, 3, 4 and 6. Nothing else started a program with a name from the list, so the rule produced no noise and needed no tuning in this version. The sample is small: one boot and a short period of normal use.

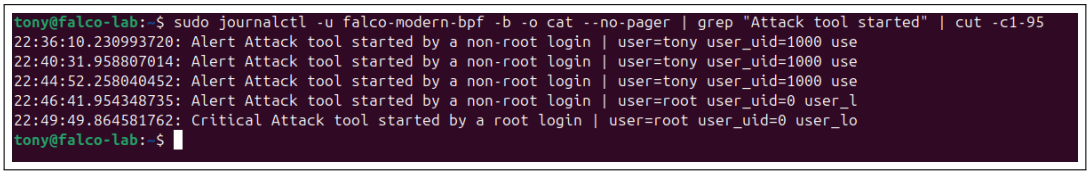



**Figure 42:** Alerts of the attack-tool rules: only the tests

### Limits

- **A renamed tool escapes the rule.** The lists match exact names, as the control `johnx` showed, so a real tool copied under another name is not seen. This is the same weakness as in the scanner rule.
- **A tool started through an interpreter can escape.** A tool written in a scripting language may start as the interpreter, and the process name would then be the name of the interpreter. This follows from how the rule works and was not tested.
- **The rule detects the start and not the attack.** The alert does not say whether anything was cracked or exploited. The command line shows the options, and the target when it is given.
- **Tools started on another machine are not seen,** only tools that start on the monitored host.
- **The login id can be changed by a privileged process.** In test 6 the login id of one shell was set to 0 by hand, and an attacker who already has root could do the same to lower the alert from `ALERT` to `CRITICAL`.
- **Some people use these tools legitimately,** such as penetration testers. The policy fits a locked-down machine and would be noisy in a security lab.
- **Most tests used stand-ins.** Only one real tool, `hydra`, was installed and started, and only with its help option.

# 9 Test scenarios

Each scenario is run only in the isolated virtual machine, with a harmless action, and is documented with its goal, the commands, the expected alert, the observed alert, and an interpretation.

## 9.1 Scenario 1: reading a sensitive file

**Goal.** Show that Falco detects the reading of the password hash file `/etc/shadow` by a program that is not on the rule's trusted list. This test was also used as the control after the tuning, to prove that the zero alerts in the measurement were real.

**Safety.** The action only reads a file, on the author's own virtual machine. The output is thrown away with `> /dev/null`, so no password hashes appear on the screen or in the evidence.

```
uptime -p
sudo cat /etc/shadow > /dev/null
sleep 3
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep "non-trusted program" | tail -3
```

- `uptime -p` shows how long the machine has been running. It confirmed that the test ran in the fresh boot (5 minutes).
- `sudo cat /etc/shadow` reads the file as the administrator. `> /dev/null` discards the output.
- `sleep 3` gives Falco time to write the alert.
- The last command shows the last three alerts of the rule in this boot. `tail -3` keeps the end of the output.

**Expected alert.** The rule *Read sensitive file untrusted* fires with priority `WARNING`, with `process=cat` and `file=/etc/shadow`.

**Observed alert.**

```
10:53:23.473440421: Warning Sensitive file opened for reading by non-trusted program | file=/etc
    /shadow gparent=sudo ggparent=bash gggparent=gnome-terminal- evt_type=openat user=root
    user_uid=0 user_loginuid=1000 process=cat proc_exepath=/usr/bin/cat parent=sudo command=cat
    /etc/shadow terminal=34817 container_id=host container_name=host container_image_repository=
     container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

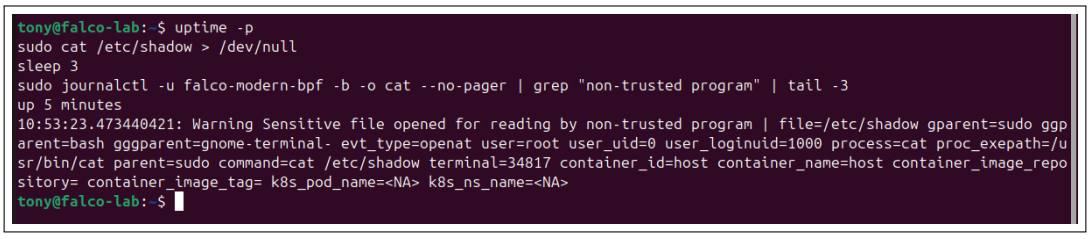



**Figure 43:** Alert raised by reading /etc/shadow with cat

**Interpretation.** The alert is a true positive: a program that is not on the trusted list read a sensitive file. Comparing it with the false positive from the login manager shows how the fields separate the two cases.

| Field | Login manager (benign) | cat test (deliberate) |
|---|---|---|
| process | gdm-session-wor | cat |
| proc_exepath | /usr/libexec/ gdm-session-worker | /usr/bin/cat |
| Parent chain | gdm3, then systemd | sudo, then bash, then gnome-terminal |
| terminal | 0, no terminal attached | 34817, typed in a terminal |
| user / user_loginuid | root / 1000 | root / 1000 |

**Table 33:** The two alerts of the same rule compared

- Both alerts show user=root, but user_loginuid=1000 shows the account that originally logged in. sudo changes the user to root, while the login identifier still points to the lab user. Falco can therefore tell that an action that looks like root started from a normal account.
- The context separates the two cases: a system program started by the login manager without a terminal, against a command typed by a person in a terminal through sudo.
- The alert after the tuning also shows that the exception is narrow: it silenced one program and not the rule.

## 9.2 Scenario 2: a shell started by a protected application

**Goal.** Show that Falco detects a shell that was started under an application that is never meant to start shells, using the default rule *Run shell untrusted*.

**The rule.** The rule is defined at line 577 of the default rules file and has the priority NOTICE. Its condition says: a program was started (spawned_process), the program is a shell (shell_procs), one of its ancestors is on the protected list (protected_shell_spawner), and none of a long list of exceptions applies. The protected list contains web servers (nginx, apache2, httpd, lighttpd), databases (mysqld, postgres), redis-server, and mail servers. The idea is that such applications serve requests and have no reason to start a shell. If one does, the application was probably taken over, which is called remote code execution. The description of the rule itself states that it can be noisy and should be tuned.

**Setup and safety.** The lab has no web server, and none is installed. The rule only checks the *name* of the ancestor, so a harmless stand-in is enough: a copy of bash saved under the name lighttpd. Nothing is installed, nothing uses the network, and the copy is deleted at the end. The process chain of the test is:

```
gnome-terminal
  bash                  (the normal shell of the user)
    lighttpd            (copy of bash, pretending to be a web server)
      sh                (the shell started by the "server")
```

```
cp /bin/bash /tmp/lighttpd
/tmp/lighttpd -c 'sh -c "echo hello from a shell"; echo done'
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep "Shell spawned by untrusted
    binary" | tail -2
rm /tmp/lighttpd
```

- cp /bin/bash /tmp/lighttpd copies the shell to a file with a protected name.

- `/tmp/lighttpd -c '...'` runs the copy, and `-c` tells it to execute the text that follows.
- `sh -c "echo hello from a shell"` is the shell started by the pretend server. It only prints a line.
- `; echo done` is a second command. Without it, `bash` would replace itself with the first command and no child shell would be started.
- The `journalctl` line keeps the last two alerts whose text contains the start of the rule's output message. `tail -2` keeps the newest.
- `rm` deletes the copy afterwards.

The commands printed `hello from a shell` and `done`.

**Expected alert.** The rule *Run shell untrusted* fires with the priority `NOTICE`, with `process=sh` and `parent=lighttpd`.

**Observed alert.**

```
12:46:36.136566801: Notice Shell spawned by untrusted binary | parent_exe=/tmp/lighttpd
    parent_exepath=/tmp/lighttpd pcmdline=lighttpd -c sh -c \"echo hello from a shell\"; echo
    done gparent=bash ggparent=gnome-terminal- aname[4]=systemd aname[5]=systemd aname[6]=<NA>
    aname[7]=<NA> evt_type=execve user=tony user_uid=1000 user_loginuid=1000 process=sh
    proc_exepath=/usr/bin/dash parent=lighttpd command=sh -c echo hello from a shell terminal
    =34816 exe_flags=0 container_id=host container_name=host container_image_repository=
    container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

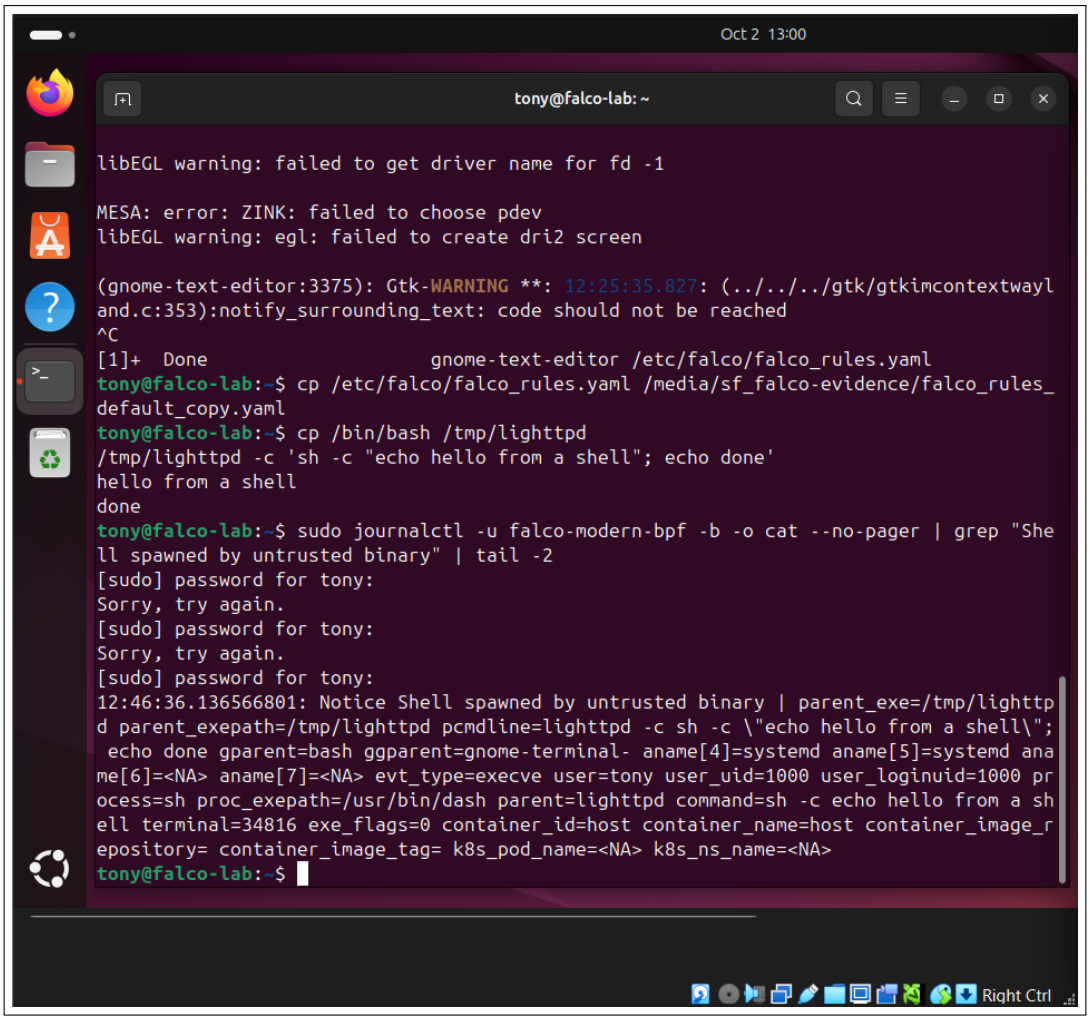



**Figure 44:** Alert raised by a shell started under a process named lighttpd

**Interpretation.** The alert is a true positive: a shell was started below a protected application name. The alert carries enough context for an analyst: the location of the parent, its command line, and the chain of processes above it.

| Aspect | Read sensitive file untrusted | Run shell untrusted |
|---|---|---|
| Behaviour detected | A sensitive file is read | A shell is started below a protected application |
| System call | `openat` | `execve` |
| Decision based on | The program that reads the file | The ancestors of the shell |
| Priority | `WARNING` | `NOTICE` |
| Test action | `sudo cat /etc/shadow` | A copy of `bash` named `lighttpd` starting `sh` |

**Table 35:** The two default rules tested so far

| Field | Value | Meaning |
|---|---|---|
| Priority | `Notice` | Lower than the `Warning` of the first rule: the rule authors rate a shell under an application as less urgent than reading the password file. |
| `evt_type` | `execve` | The system call that starts a program. The first rule watched `openat`, which opens a file. |
| `process` | `sh` | The shell that was started, which is what the rule is about. |
| `proc_exepath` | `/usr/bin/dash` | The real file behind the name `sh`. On Ubuntu, `sh` is a link to `dash`. |
| `parent` | `lighttpd` | The pretend server that started the shell. |
| `parent_exepath` | `/tmp/lighttpd` | The “server” runs from `/tmp`. A real web server runs from a system folder, so this is suspicious on its own. |
| `pcmdline` | `lighttpd -c sh -c ...` | What the parent was told to do. In a real attack it would reveal the attacker’s command. |
| `gparent`, `ggparent`, `aname[4]` | `bash`, `gnome-terminal-`, `systemd` | The chain above the parent: the terminal started the shell of the user, which started the pretend server. |
| `user` / `user_loginuid` | `tony` / `1000` | An ordinary user, not root. |
| `terminal` | `34816` | A terminal is attached, because the command was typed by hand. |
| `container_id` | `host` | The event happened on the machine and not inside a container. |

**Table 34:** Fields of the second alert

**Limits of this test.**

- The test simulates the behaviour of an attack but is not identical to one. In a real attack the web server would have no terminal (`terminal=0`), because nobody types the command at a keyboard.
- In the log of this boot, the rule raised only this alert, so normal use of the desktop produced no noise for this rule. This is one boot, a small sample. The description of the rule warns that it can be noisy in other environments, and the default file provides the empty list `user_known_shell_spawn_binaries` for local exceptions. No exception was needed here.

## 9.3 Scenario 3: a program run from /dev/shm

**Goal.** Show that Falco detects a program that is started from the directory `/dev/shm`, using the default rule *Execution from /dev/shm*.

**The rule.** The rule is defined at line 1172 of the default rules file and has the priority `WARNING`. Its condition says: a program was started (`spawned_process`) and one of these is true: the path of the program starts with `/dev/shm/`, the program was started with a `./` path from inside that directory, or a shell was told to run a script from there. Programs of a few trusted container images are excluded. The tags include `mitre_execution` and `T1059.004` (the MITRE sub-technique *Unix Shell*).

**Why this behaviour matters.** The directory `/dev/shm` lives in memory and not on the disk. Any user can write to it, and its content disappears at reboot, so a file placed there leaves almost no trace on the disk. This makes it attractive for an attacker who has just gained access to a machine and needs a place to put and run a tool: the attacker saves the tool there and starts it. Normal software uses the directory to share data and does not start programs from it, so a program started from `/dev/shm` is a strong warning sign. Together with the two earlier rules, it covers three stages of an intrusion: reading credentials, taking control with a shell, and running the attacker's tool.

**Setup and safety.** The test plays the last two steps of the attacker with a harmless program: a copy of `echo`, which only prints the words it is given, is saved in `/dev/shm` and started from there. Nothing is installed and nothing uses the network. The file lives in memory and is deleted at the end.

```
systemctl is-active falco-modern-bpf
cp /bin/echo /dev/shm/hello_shm
ls -l /dev/shm/hello_shm
/dev/shm/hello_shm "hello from /dev/shm"
sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep "File execution detected from /
    dev/shm" | tail -2
rm /dev/shm/hello_shm
```

- `systemctl is-active` prints one word about a service. It confirmed that Falco was running, so that a missing alert could not be caused by a stopped service.
- `cp /bin/echo /dev/shm/hello_shm` copies the program into the memory directory under a new name. Copying is not running, so no alert is expected at this point.
- `ls -l` lists the file with its details. The `x` in the permissions shows that it may be run, and the owner is the ordinary user, who created it without administrator rights.
- `/dev/shm/hello_shm "hello from /dev/shm"` starts the copy by its full path, which begins with `/dev/shm/`. This is the action the rule watches.

- The `journalctl` line keeps the last two alerts whose text contains the start of the rule's output message.
- `rm` deletes the copy afterwards.

The commands printed `active`, then the line of `ls -l` (a file of 35208 bytes with the permissions `-rwxr-xr-x`, owned by the lab user), and then `hello from /dev/shm`.

```
tony@falco-lab:~$ systemctl is-active falco-modern-bpf
active
tony@falco-lab:~$ cp /bin/echo /dev/shm/hello_shm
tony@falco-lab:~$ ls -l /dev/shm/hello_shm
-rwxr-xr-x 1 tony tony 35208 Oct  2 21:02 /dev/shm/hello_shm
tony@falco-lab:~$ /dev/shm/hello_shm "hello from /dev/shm"
hello from /dev/shm
```

**Figure 45:** Steps of the test: Falco active, copy, check, and run

**Expected alert.** The rule *Execution from /dev/shm* fires with the priority `WARNING`, with `proc_exepath=/dev/shm/hello_shm`.

**Observed alert.**

```
21:07:07.855742446: Warning File execution detected from /dev/shm | evt_res=SUCCESS file=<NA>
    proc_cwd=/home/tony/ proc_pcmdline=bash user_loginname=tony group_gid=1000 group_name=tony
    evt_type=execve user=tony user_uid=1000 user_loginuid=1000 process=hello_shm proc_exepath=/
    dev/shm/hello_shm parent=bash command=hello_shm hello from /dev/shm terminal=34816 exe_flags
    =EXE_WRITABLE container_id=host container_name=host container_image_repository=
    container_image_tag= k8s_pod_name=<NA> k8s_ns_name=<NA>
```

```
tony@falco-lab:~$ sudo journalctl -u falco-modern-bpf -b -o cat --no-pager | grep "File execution detected from /dev/shm
" | tail -2
[sudo] password for tony:
Sorry, try again.
[sudo] password for tony:
21:07:07.855742446: Warning File execution detected from /dev/shm | evt_res=SUCCESS file=<NA> proc_cwd=/home/tony/ proc_
pcmdline=bash user_loginname=tony group_gid=1000 group_name=tony evt_type=execve user=tony user_uid=1000 user_loginuid=1
000 process=hello_shm proc_exepath=/dev/shm/hello_shm parent=bash command=hello_shm hello from /dev/shm terminal=34816 e
xe_flags=EXE_WRITABLE container_id=host container_name=host container_image_repository= container_image_tag= k8s_pod_nam
e=<NA> k8s_ns_name=<NA>
tony@falco-lab:~$
```

**Figure 46:** Alert raised by a program started from /dev/shm

**Interpretation.** The alert is a true positive: a program was started from a directory that normal software does not run programs from. The alert tells an analyst what ran, from which path, who started it, with which command line, and whether the start succeeded.

| Rule | Stage of an intrusion | System call | Priority |
|---|---|---|---|
| Read sensitive file untrusted | Stealing credentials: a program reads the password files | `openat` | `WARNING` |
| Run shell untrusted | Taking control: a shell starts below a server | `execve` | `NOTICE` |
| Execution from /dev/shm | Running the attacker's tool: a dropped program starts | `execve` | `WARNING` |

**Table 37:** The three default rules tested

| Field | Value | Meaning |
|---|---|---|
| Priority | `Warning` | The same level as the first rule and higher than the `Notice` of the shell rule. |
| `evt_type` | `execve` | The system call that starts a program, as in the shell rule. |
| `evt_res` | `SUCCESS` | The program really ran. The result field separates a successful start from a failed attempt. |
| `process` / `proc_exepath` | `hello_shm` / `/dev/shm/hello_shm` | The program and its full path. The path begins with `/dev/shm/`, which is what the rule matches. |
| `proc_cwd` | `/home/tony/` | The directory the user was in. It is the home directory, so the rule matched on the path of the program. |
| `parent` / `proc_pcmdline` | `bash` / `bash` | The shell of the user started the program. |
| `command` | `hello_shm hello from /dev/shm` | The command line, with the words that were passed. |
| `user` / `user_loginuid` | `tony` / `1000` | An ordinary user. The rule watches the location of the program and not the person. |
| `terminal` | `34816` | A terminal is attached, because the command was typed by hand. |
| `exe_flags` | `EXE_WRITABLE` | A field that Falco adds to alerts about started programs. `EXE_WRITABLE` marks an executable file that is writable. The test file was owned by the ordinary user, while the system shell in Scenario 2 showed `0`. |
| `container_id` | `host` | The event happened on the machine and not inside a container. |

**Table 36:** Fields of the third alert

**Limits of this test.**

- The test simulates only the moment when the tool is started. A real attack would also download the tool, and the program would not be a harmless `echo`.
- In the log of this boot, the rule raised only this alert, so normal use of the desktop produced no noise for this rule. This is one boot, a small sample, and no tuning was needed.
- An attacker can use another location, such as `/tmp` or a home directory, and this rule would not see it. This gap is a reason to write a custom rule for another location.

### 9.4 Scenario 4: the setuid bit (custom rule A)

The fourth scenario is the test of the first custom rule. Its commands, alerts, noise and tuning are documented with the rule in Section 8.1.

### 9.5 Scenario 5: a network scanner (custom rule B)

The fifth scenario is the test of the second custom rule: a harmless stand-in named like a network scanner is started by an ordinary account, with `sudo`, as a service, and as a simulated root login. Its commands, alerts, noise and limits are documented with the rule in Section 8.2.

### 9.6 Scenario 6: tampering with Falco (custom rule C)

The sixth scenario groups the tests of the four rules that watch attempts to blind Falco: a write to its files, a refused write, a rename or deletion, and the stopping of its service. The tests use harmless files and stand-ins, and the real service was stopped for about three seconds. They are documented with the rules in Section 8.3.

### 9.7 Scenario 7: attack tools (custom rule D)

The seventh scenario is the test of the fourth custom rule: stand-ins named like a password-cracking tool and a break-in toolkit, and one real tool started with its help option, are started by an ordinary account, with `sudo`, and from a simulated root login. They are documented with the rule in Section 8.4.

## 10 Alert quality, false positives and tuning

### 10.1 Baseline

Before tuning, the rule *Read sensitive file untrusted* produced 51 alerts in the whole Falco log, all from one program, the GNOME login manager session worker (Table 9 in the default rules review). Verification of the program against its package showed that these alerts are false positives.

## 10.2 Controlled comparison

With the same procedure (one boot and one login), the rule produced 27 alerts before the exception and 0 after it (Table 12). A deliberate read of `/etc/shadow` with `cat` after the tuning still raised the alert, so the reduction did not come from losing the detection. The limits of this measurement are listed in the default rules review.

## 10.3 Alert quality table

| Rule | Situation | Expected alert | Observed alert | Verdict |
|---|---|---|---|---|
| *Read sensitive file untrusted* | Desktop login (login manager reads PAM and shadow files) | None, benign activity | 27 alerts before tuning, 0 after | False positive, removed by the exception |
| *Read sensitive file untrusted* | `sudo cat /etc/shadow` | Alert | Alert raised, `WARNING` | True positive |
| *Run shell untrusted* | Shell started by a process named `lighttpd` | Alert | Alert raised, `NOTICE` | True positive |
| *Execution from /dev/shm* | Program started from `/dev/shm` | Alert | Alert raised, `WARNING` | True positive |
| *Setuid or setgid bit set on a file* | `chmod u+s`, `chmod 4600`, `chmod 2600` on a file | Alert | Alert raised, `WARNING` | True positive |
| *Setuid or setgid bit set on a file* | Reinstall of a package (`apt install -reinstall fuse3`) | None, benign activity | 2 alerts before tuning, 0 after | False positive, removed by the exception |
| *Network scanner started by a non-root login* | Stand-in named `nmap` started by the lab user, with `sudo`, or as a service | Alert | Alert raised, `CRITICAL` | True positive |
| *Network scanner started by a root login* | The same stand-in started from a simulated root login | Alert | Alert raised, `WARNING` | True positive |
| *Network scanner* (both rules) | A normal command and the lookalike `nmapx` | None | Silent | True negative |

**Table 38:** Alert quality of the default rules and of custom rules A and B

| Rule | Situation | Expected alert | Observed alert | Verdict |
|---|---|---|---|---|
| *Falco configuration or rules file written* (C2) | A write to the rules file through `sudo` | Alert | Alert raised, `WARNING` | True positive |
| *Falco configuration or rules file written* (C2) | One `nano` edit | One alert for the real file | 3 alerts before the exception, 1 after | Noise (swap file), removed by the exception |
| *Denied write attempt on a Falco file* (C1) | A write by an ordinary user, refused | Alert | Alert raised, `CRITICAL`, `EACCES` | True positive |
| *Falco file renamed or deleted* (C3) | A file moved into, and deleted from, `/etc/falco/` | Alert | Alert raised, `WARNING`; C2 saw nothing for the move | True positive |
| *Falco file renamed or deleted* (C3) | An `nano` session with nothing saved | None, or an audit trail | One alert (the swap file is deleted) | Kept on purpose as an audit trail |
| *Falco service stopped or disabled* (C4) | A stand-in `systemctl`, and a real `stop` | Alert | Alert raised, `ALERT` | True positive |
| *Falco service stopped or disabled* (C4) | `systemctl status` and `restart` | None | Silent | True negative |

**Table 39:** Alert quality of custom rule C

| Rule | Situation | Expected alert | Observed alert | Verdict |
|---|---|---|---|---|
| *Attack tool started by a non-root login* (D) | Stand-ins named `john` and `msfconsole`, and the real `hydra` with its help option, started by the lab user | Alert | Alert raised, `ALERT` | True positive |
| *Attack tool started by a non-root login* (D) | The stand-in `john` with `sudo` | Alert | Alert raised, `ALERT`; the login id stays 1000 | True positive |
| *Attack tool started by a root login* (D) | The same stand-in from a simulated root login | Alert | Alert raised, `CRITICAL` | True positive |
| *Attack tool started* (D, both rules) | A normal command and the lookalike `johnx` | None | Silent | True negative |

**Table 40:** Alert quality of custom rule D

## 10.4 Noise and tuning

The noise came from one verified program that the rule did not know. It was removed with a narrow exception in the local rules file, using the full path of the program and the hook that the rule provides, which kept the detection for every other program.

The second rule, *Run shell untrusted*, produced no alert during normal use of the desktop in the boot that was observed, so it needed no tuning in this lab. Its description warns that it can be noisy elsewhere.

The third rule, *Execution from /dev/shm*, also produced no alert during normal use of the desktop in the boot that was observed, so it needed no tuning in this lab.

The custom rule *Setuid or setgid bit set on a file* produced noise from package installs. The alert showed that the bit was set by `chmod`, run by the post-install script of the package, with `dpkg` and `apt` above it. An exception was added for the case where a package manager is in the chain and the file is in a system folder, and the same reinstall then stayed silent (count 5 before and 5 after). Controls showed that the rule still alerts for a bit set in `/tmp`, even with a stand-in package manager above it, and a limit was found in the folder test (see the limits of the rule).

The two scanner rules of custom rule B produced no noise: after about an hour and twenty minutes of normal desktop use, the only alerts were the four caused by the tests, so no tuning was needed in this version.

The rules of custom rule C produced two kinds of noise. C2 raised three alerts for one `nano` edit, because of the swap file, and an exception for the swap file of `nano` reduced this to one (the count of alerts went from 8 to 9 for one edit). C3 raises one alert when `nano` deletes its swap file on exit, even if nothing was saved, and this was kept on purpose as an audit trail. C1 and C4 raised no alert during normal use.

The two rules of custom rule D produced no noise: the only alerts in the boot were the five caused by the tests, so no tuning was needed in this version.

# 11 Decisions and lessons learned

## 11.1 Decisions

| Decision | Reason |
|---|---|
| Ubuntu 24.04 LTS instead of 26.04.1 | Risk management: 24.04 is the better established release for Falco and its tools. The upgrade prompt to 26.04.1 was declined. |
| Check the Falco version before installing | The kernel 7.0 problem reported for Falco 0.44.x made the version critical. |
| Modern eBPF driver | Safest of the three drivers and needs no kernel-specific build. |
| Snapshot before the installation | Gives a rollback point if the installation or a later change fails. |
| Tune with the hook macro and an exact program path | Narrowest way to remove the false positive: one verified program is ignored and the detection for all others is kept. |

**Table 41:** Decisions taken while building the lab

## 11.2 Lessons learned

- **Check compatibility before installing a kernel-level tool.** Reading the issue tracker and comparing versions took a few minutes and avoided a failure that would only appear after some time of running.
- **The newest release is not automatically the safest.** Both the newer Ubuntu release and the newer kernel carried risk, and the decision to stay on a well-established release reduced it.
- **A finished installation is not proof that a tool works.** The service status and the restart counter give a first indication, and a longer test under activity is needed to confirm stability, and this was not done in this project.
- **Diagnose step by step.** The clipboard problem was solved by checking in turn whether the helper was running, whether the modules were loaded, and then starting the helper by hand.
- **Group the data by the right field.** Counting alerts per file suggested several sources of noise, but counting them per program showed that all 51 alerts came from one program. The first grouping pointed to the wrong conclusion.
- **Check the exact characters of a command.** A search pattern with one missing space returned no output, which looks the same as finding nothing. An empty result has to be checked before it is believed.
- **Validate before restarting.** Checking the rules files with `falco -V` found no problem before the running service was touched, so a mistake could not have broken it.

- **Do not read a status immediately after a restart.** The first reading, taken 10 milliseconds after the restart, showed a process that had not finished starting. The meaningful reading came after waiting.
- **A zero needs a control.** A count of zero alerts could mean that the exception works or that nothing is being detected. A deliberate trigger that still raised the alert showed which one it was.
- **Tune at the narrowest point.** Removing a folder from the definition of sensitive files would also have removed the false positive, but it would have hidden real reads of those files. An exception for one verified program does not.
- **Test a rule with a harmless stand-in.** The shell rule checks the name of the parent program, so a copy of a shell saved under the name of a web server was enough to trigger it, without installing a server or attacking anything.
- **Document while working.** A command log kept from the first session makes the deployment reproducible.
- **A test can reproduce only the step that a rule watches.** The `/dev/shm` rule watches the moment a program is started, so a harmless program started from that directory was enough, without simulating the rest of an attack. The limits of such a test have to be stated.
- **Examples for older versions can be out of date.** A condition copied from older examples used a field that this version of Falco no longer needs, and the validation warning showed it.
- **Read the whole chain before writing an exception.** The expected process was the package manager, but the process that set the bit was `chmod`, run by an install script, and only the ancestors identified the context.
- **Combine conditions to keep an exception narrow.** Requiring both a package manager above the program and a system folder kept the alerts for files in `/tmp` and similar places.
- **A rule that compares text can be fooled by a path that means something else.** A path that starts in a system folder but climbs out of it with `..` was not caught, and finding such limits needs deliberate tests.
- **Choose the field that answers the question.** Both user fields exist, but only the login id identifies the person behind a `sudo` command, and a test with `sudo` showed the difference.
- **Decide the policy before writing the rule.** A rule has one priority, so a policy with two levels became two rules that share one list.
- **A control with a lookalike name shows that a rule is exact,** and also shows how a renamed tool escapes it.
- **A policy based on an identifier that a privileged process can change has a limit.** Setting the login id by hand lowered the alert, and finding this required testing it.
- **Count what your own actions trigger.** The count of alerts of C2 was 6 instead of 3, because the editor session that added another rule had raised alerts too, and the full listing explained every one.
- **A default macro can hide a case.** `open_write` only matches opens that succeeded, so refused attempts needed their own rule.
- **Test the gap before writing the rule.** A file moved into Falco's folder raised no alert from C2 (0 before and 0 after), and that result justified C3.
- **Use the ladder of priorities with a reason for each step.** `ALERT` was kept for the two cases that need an immediate answer: the monitor being switched off (C4) and an attack tool started by an ordinary account (D).
- **Confirm a stand-in with one real tool.** A copy of a harmless program named like an attack

tool tests a rule that reads names, and one real tool, started only with its help option, gave the same alert, which shows that the stand-in is a fair test.

# 12 Conclusion

A Falco 0.45.0 lab was built on an Ubuntu 24.04 LTS virtual machine with kernel 7.0.0-34 and the modern eBPF driver. The version was chosen after a compatibility check, because Falco 0.44.x is reported to stop on 7.0 kernels and the fix ships in 0.45.0. Three default rules were examined from real alerts and tested, and four detection ideas of my own were written as nine custom rules, tested and, where needed, tuned (Table 42).

| Rule | Behaviour | Result |
|---|---|---|
| Default: read sensitive file untrusted | A program reads a password file | 27 alerts from the login manager before tuning, 0 after; the control still alerted |
| Default: run shell untrusted | A shell starts below a server | True positive, no noise |
| Default: execution from `/dev/shm` | A program starts from a memory directory | True positive, no noise |
| Custom A | The setuid or setgid bit is set on a file | Package installs raised noise, removed by an exception on the package manager and the system folders |
| Custom B (two rules) | A network scanner starts on the host | Two tiers by the login id; no noise |
| Custom C (four rules) | Falco is blinded: files written, refused, renamed or deleted, service stopped | Noise from the swap file of `nano` reduced from 3 alerts to 1; the alert for a real stop was written before Falco stopped |
| Custom D (two rules) | An attack tool starts: password cracking or break-in toolkits | Two tiers by the login id (`ALERT` and `CRITICAL`); no noise; one real tool confirmed the stand-ins |

**Table 42:** Summary of the rules and their results

## 12.1 What the work showed

- **A rule that works is not finished.** The first default rule fired on a harmless program, the login manager. Verifying the program, writing the narrowest exception and measuring the effect turned it into a useful rule, and the same method was applied to the later rules.
- **Every result needs a control.** A count of zero could mean that the tuning works or that nothing is detected, and a silent test could mean that the rule is broken. Deliberate triggers that still alerted separated the two.
- **Alerts have to be read, not only counted.** Reading the chain of processes showed that a package install was the cause of the noise of custom rule A, and listing the alerts of custom rule C showed that one editor session raised three of them.

- **Testing finds the limits.** Several limits were found by testing them and not by reading about them: a rule that matches names is evaded by a renamed program, a rule that compares the start of a path can be passed by a path that climbs out of the folder, a rename into a folder raises no alert from a rule that watches writes, and a privileged process can change its own login id.

### 12.2 Limits of the project

The work used one virtual machine and small samples, mostly one boot of normal desktop use, so the absence of noise is an observation and not a guarantee. Harmless stand-ins replaced real tools in most tests (a copy of `bash` named like a web server, a copy of `true` named like a scanner or an attack tool), so those tests check the rules and not the tools; one real tool, `hydra`, started only with its help option, gave the same alert as its stand-in. No dedicated stability test was run, and the output settings and the update helper `falcoctl` were not reviewed. Falco is also a detection system and not a prevention system: the alert comes after the event, and nothing is blocked.

### 12.3 Possible extensions

- Send the alerts off the machine, so that an attacker with root cannot erase them, and test whether the alert for a stopped service always arrives.
- Close the path limit of custom rule A, for example by using the field that holds the fully resolved path, and test it with the same tricky path.
- Extend the rules that watch Falco's own files to other editors, and review the writes of `falcoctl`.
- Measure the noise of `ss` and `netstat` before adding them to the scanner list, and look at tools that are started through an interpreter, which the name-based rules of B and D can miss.
- Repeated failed logins were considered and not built: they need a count over many events, which a Falco rule does not do, and a tool that reads the authentication log, such as `fail2ban` or `auditd`, suits that job better.

Falco is a good detector of behaviour and a poor judge of intent. Its value comes from rules that are fitted to the environment, tested with controls, and written together with their limits.

## 13 References

### Falco


- Falco documentation, *Controlling Rules*: `https://falco.org/docs/concepts/rules/controlling-rules/`
- Falco documentation, *Overriding Rules*: `https://falco.org/docs/concepts/rules/overriding/`
- Falco documentation, *Style Guide of Falco Rules*: `https://falco.org/docs/concepts/rules/style-guide/`
- Falco documentation, *Accessing File System Paths in Falco Rules*: `https://falco.org/docs/concepts/rules/fspath/`

- Falco documentation, *Supported Fields for Conditions and Outputs*: https://falco.org/docs/reference/rules/supported-fields/
- Falco issue #3955 (Falco 0.44.x on Linux 7.0 kernels): https://github.com/falcosecurity/falco/issues/3955

## MITRE ATT&CK

- T1555, *Credentials from Password Stores*: https://attack.mitre.org/techniques/T1555/
- T1003.008, *OS Credential Dumping: /etc/passwd and /etc/shadow*: https://attack.mitre.org/techniques/T1003/008/
- T1548.001, *Abuse Elevation Control Mechanism: Setuid and Setgid*: https://attack.mitre.org/techniques/T1548/001/
- T1059.004, *Command and Scripting Interpreter: Unix Shell*: https://attack.mitre.org/techniques/T1059/004/
- T1046, *Network Service Discovery*: https://attack.mitre.org/techniques/T1046/
- T1562.001, *Impair Defenses: Disable or Modify Tools*: https://attack.mitre.org/techniques/T1562/001/